

队伍编号	202502997
选题	A 题

康养城市建设

摘 要

2025 年全国两会期间，多位代表和委员围绕康养城市建设、智慧养老、医养融合等议题提出了重要建议。康养城市建设不仅是国家层面的战略部署，也是地方发展的实际需求。本文初步解决了，如何在有限的资源条件下，优化康养设施布局，提升康养服务质量，推动智慧养老与康养产业的深度融合。

对于问题一，针对某城市资源合理配置，首先由于数据的不可获取性，利用**蒙特卡洛**模拟获取相关数据；其次，利用模拟数据构建资源指数与健康指数，分别使用熵值法与**模糊综合评价法**求解，在此基础上进行资源-健康指数相关性分析，判断出当前康养资源分布并不合理；最后，以最大化总健康指数为目标函数，以资源总量固定、每个区域至少获得一定资源为约束条件，构建目标优化模型，得到优化后整体总健康指数显著增加，为资源分配提供参考价值。

对于问题二，采用**熵权法与层次分析法**结合确定指标权重，然后结合**TOPSIS 法**从康养资源丰富度、居民健康状况、环境质量、经济发展水平等四个维度设定评价指标建立综合评价模型并计算综合得分，引入**灰色关联分析**，得到各城市与理想城市的距离，分析结果并提出合理性建议。

对于问题三，针对康养城市资源优化配置，首先构建多目标函数优化模型，以最大化康养服务覆盖率、最小化成本投入、最大化资源利用率为目标，将设施选址、流动车部署、服务量分配等作为决策变量，设置容量约束、预算约束等条件。然后采用**粒子群优化算法**求解，得到资源利用率 82.47%。最后结合模型结果与问题 1 研究成果，提出涵盖政策建议、资金投入方向、技术创新措施等具体实施策略，为政府制定康养城市发展规划提供科学依据。

关键词 **蒙特卡洛 模糊综合评价法 TOPSIS 法 灰色关联分析 粒子群优化算法**

一、问题重述

如何在有限的资源条件下，优化康养设施布局，提升康养服务质量，推动智慧养老与康养产业的深度融合，是亟待解决的问题。请搜集相关数据，建立评价指标体系，完成以下问题。

问题 1：康养资源分布的现状分析与优化需求

康养城市的建设离不开对现有康养资源的全面评估。请收集或使用模拟数据分析某城市内不同区域的康养资源分布情况包括医疗设施、养老机构、公园绿地、文化设施等的数量和分布密度。同时，结合居民健康状况数据（如平均寿命、慢性病发病率、健康满意度等），分析当前康养资源分布的合理性。在此基础上，提出优化康养资源布局的初步思路，重点关注如何解决资源分布不均衡的问题，以提升康养服务的公平性和可达性。

问题 2：康养城市综合评价模型的构建与应用

康养城市的建设需要科学的评价体系来衡量其发展水平。请构建一个康养城市的综合评价模型，从康养资源丰富度、居民健康状况、环境质量、经济发展水平等多个维度设定评价指标，并通过模型计算，分析当前城市康养水平的优势与不足，并提出针对性的改进建议，为康养城市建设提供科学依据。

问题 3：康养资源优化配置模型的建立与实施策略

在有限的资源条件下，如何高效配置康养资源是康养城市建设的关键。请根据综合评价结果，建立一个康养资源优化配置模型，目标是最大化康养服务的覆盖率和资源利用效率，同时最小化成本。设计多目标优化函数，考虑康养设施的选址、数量、服务范围等因素，并使用线性规划或其他优化算法求解模型。最后，结合模型结果和问题 1 的研究成果，提出具体的实施策略，包括政策建议、资金投入方向、技术创新措施等，为政府制定康养城市发展规划提供依据。

二、问题分析

2.1 问题一的分析

康养城市的建设离不开对现有康养资源的全面评估。首先，某城市各区域的康养资源与居民健康状况难以获取，为此通过蒙特卡洛模拟获取相关数据，如医疗设施数、绿地面积、慢性病获病率等向量；其次，为判断资源配置的合理性，直接对比以上向量缺乏实际意义，故以建立资源指数与健康指数为基础，利用皮尔逊相关系数构建资源-健康指数相关性分析，判断资源配置的合理性。其中，在计算健康指数时，由于其中中存在居民健康满意度指标，为此利用模糊综合评价方法进行求解。最后，利用多项式回归模型建立资源指数与健康指数的关系，并构建基尼指数作为公平性评估，以此为基础构建目标优化模型，进行资源优化配置。

2.2 问题二的分析

问题二旨在构建一个科学的康养城市综合评价模型，以全面衡量城市在康养资源丰富度、居民健康状况、环境质量和经济发展水平等多个维度的表现。首先根据各个维度下的指标计算各维度的指数。然后，通过结合熵权法与层次分析法确定各维度的权重，既保证数据的客观性，又融入专家的主观判断，提高权重分配的合理性。随后，利用 TOPSIS 法计算各城市与理想状态的距离，实现对城市康养水平的排序和综合评价。引入灰色关联分析则进一步揭示各城市与理想城市在具体指标上的差距，帮助精准识别优势和短板。整体模型不仅能定量反映康养城市的发展现状，还为相关部门提出针对性的改进建议提供科学依据，助力推动康养城市高质量发展。

2.3 问题三的分析

在康养城市建设中，资源优化配置需破解公平性、效率性与成本控制的核心矛盾。问题三通过构建多目标函数优化模型，系统性整合设施选址、流动车部署、服务量分配等决策变量，以最大化康养服务覆盖率、最小化成本投入、最大化资源利用率为目标，平衡区域需求差异与财政约束。模型引入容量约束、预算约束及关联性约束，确保方案可行性。采用粒子群优化算法求解，通过编码粒子表征资源配置方案，动态调整粒子速度与位移，经迭代优化后实现资源利用率 82.47% 的优化效果。最终结合问题 1 的资源分布现状分析，从政策布局、资金投向、技术创新等维度提出实施策略，为政府构建“核心区固定设施主导、边缘区流动车补充”的高效资源配置模式提供科学支撑，助力提升康养服务的公平性与可及性。

三、模型假设

- 假设 1：问题一的资源总量不发生改变；
- 假设 2：每个区域的绿地面积具有最低限额，符合城市规划常数；
- 假设 3：在进行蒙特卡洛模拟时，各种资源数量服从泊松分布。

四、符号说明

符号	说明
x_{ij}	第 i 个城市下第 j 项指标的数据
C_i	第 i 个城市的 TOPSIS 得分

Γ_i	第 i 个城市的灰色关联度
y_{ij}	设施 i 为区域 j 提供的服务量占总需求的比重
s_i	设施规模量
d_{kj}	流动车 k 的部署位置到需求区域 j 的实际空间距离
$c_i^{\text{建设}}$	第 i 个固定设施的单位建设成本

五、问题一模型的建立与求解

康养城市的建设离不开对现有康养资源的全面评估。康养资源是指支持健康养老和提升居民生活质量的各种物质和服务条件，涵盖了医疗、养老、环境、文化等多个方面，为空间性资源，每个地区的资源丰富度有所差异。

5.1 数据的获取

5.1.1 城市区域划分与向量的选取

由于康养资源的空间性，以街道、社区为划分标准，将城市划分为若干个互不重叠的“子单元”，将每个单元作为分析对象，以医疗设施数、养老机构数、公园绿地面积、文化设施数、区域面积、平均寿命、慢性病发病率、健康满意度作为向量表示该区域的康养状态，其具体表示如表 1 所示：

表 1 各指标表示与具体含义

符号	名称	具体含义
M_i	区域 i 的医疗设施数	医疗设施多寡直接影响医疗服务的可达性和质量。
E_i	区域 i 的养老机构数	养老资源供给水平，关联老年群体健康管理需求。
G_i	区域 i 的绿地面积	反映区域的生态环境和居民休闲空间，间接改善健康。
C_i	区域 i 的文化设施数	文化设施丰富有助于提升居民精神文化生活质量。
A_i	区域 i 的面积	是计算资源密度、人口密度等指标的基准。
L_i	平均寿命	衡量居民健康水平，是康养服务成效的重要健康指标。
D_i	慢性病患率	反映居民健康风险和疾病负担。
S_i	居民健康满意度	体现康养服务的社会效果和居民幸福感。

5.1.2 模拟数据——蒙特卡洛模拟^[1]

由于数据难以获取，本文使用蒙特卡洛模拟技术生成城市区域数据，将城市划分为 20 个区域，每个区域模拟出上述各向量数据，涵盖康养资源与居民健康状况。其具体的模拟的分布与参数的选取如下。

1. 泊松分布——离散事件计数

泊松分布适用于单位空间内随机事件的发生次数，该分布的均值和方差相等，能体现设施分布的随机性。

应用：医疗设施、养老机构、文化设施

参数：

- (1) 医疗设施：均值 $\lambda = 10$ ，模拟中等规模城市单个区域的医疗设施数量。
- (2) 养老机构：均值 $\lambda = 5$ ，相较于养老机构，数量少，分布更为稀疏。
- (3) 文化设施：均值 $\lambda = 3$ ，文化设施密度通常低于医疗和养老设施。

2. 正态分布——连续中心趋势数据

正态分布适用于有明确中心趋势的数据。

应用：绿地面积、平均寿命、慢性病患率

参数：

- (1) 绿地面积：均值 $\lambda = 30$ 公顷，标准差为 10 公顷，截断最小值为 5 公顷。
- (2) 平均寿命：均值 $\lambda = 76$ 岁，标准差为 3，参数中国城市人口平均寿命。
- (3) 慢性病患率：均值 $\lambda = 15\%$ ，标准差为 5%，参考中国慢性病总体患病率。

3. 均匀分布——连续等概率数据

均匀分布可生成等概率的连续值，模拟不同规模的区域。

应用：区域面积

参数：

- (1) 区域面积：参数范围 $[2, 20] km^2$ ，避免区域面积过小不符合城市规划实际。

4. 离散型概率分布——离散型定性数据

应用：满意度调查

参数：

- (1) 满意度调查：参数范围[好，较好，一般，较差，差]，覆盖合理满意度区间。

根据上述分布方法与参数的选取，利用蒙特卡洛模拟得出各向量数据，部分数据如表 2 所示，完整数据内容请查看附件（为方便起见，该附件包含下文资源指数的四个指标，后文不再赘述）。

表 2 某城市康养的部分模拟数据

区域 编号	医疗设 施数	养老机 构数	绿地面积 (公顷)	文化 设施 数	区域面积 (km ²)	平均寿命 (岁)	慢性病患病 率(%)	健康满意度
R1	12	2	14.493366	3	10.38	69.9	21.3	差
R2	6	5	30.685630	3	11.77	76.6	11.5	一般
R3	11	5	19.376963	2	7.16	74	17.2	差
R4	14	6	34.735924	3	12.63	78.6	18.9	一般
R5	7	4	20.805758	2	2.55	73.6	10.4	好
R6	8	6	45.499344	6	2.67	75.7	14.7	一般

5.2 资源指数模型建立与求解——熵值法

5.2.1 资源指数模型的建立

为衡量某一地区的康养资源供给能力,将多种资源整合为一个统一的资源指数。由于直接对比“绿地数量”与“医院数量”等向量缺乏实际意义,因此设立医疗设施密度 x_i^1 (医疗设施数/区域面积)、养老机构密度 x_i^2 (养老机构数/区域面积)、绿地密度 x_i^3 (绿地面积/区域面积)、文化设施密度 x_i^4 (文化设施数/区域面积)这四个指标,用于构建资源指数,部分数据展示如下表 3 所示。

表 3 资源指数的部分指标数据

区域编号	医疗设施密度	养老机构密度	绿地密度	文化设施密度
R1	1.16	0.19	1.4	0.29
R2	0.51	0.42	2.61	0.25
R3	1.54	0.7	2.71	0.28
R4	1.11	0.48	2.75	0.24
R5	2.75	1.57	8.16	0.78
R6	3	2.25	17.04	2.25

为量化每个区域的康养资源水平,构建资源指数:

$$R_i = \sum_{j=1}^4 w_j x_i^j,$$

其中, w_j 表示第 j 类资源的权重。

5.2.2 资源指数的模型求解

1.数据标准化及一致化

以上四个指标均为极大型指标,故其处理公式为:

$$x'_i = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}},$$

其中， $x_{\min}$ 为指标 x^j 的最小值， $x_{\max}$ 为指标 x^j 的最大值。标准化后， $x'_i=1$ 表示第 i 种资源的最优。

2. 非负平移与信息熵值 e 的求解

$$x''_i = x'_i + 0.001,$$

其中，由于熵值法计算过程中需要取对数，即要求数据不能为 0，故将数据整体加 0.001，确保数据均为正值，

$$P_i^j = \frac{x''_i}{\sum_{i=1}^{20} x''_i},$$

其中，以第 j 个指标为例， P_i 表示在该指标下，第 i 个区域的资源占全部资源的比重。

$$e^j = -m \cdot \sum_{i=1}^{20} (P_i^j \cdot \ln(P_i^j)),$$

其中， $m = 1/\ln(20)$ ， e 表示指标的信息熵。

3. 信息效用值 d 与权重系数值 w 的计算

$$d^j = 1 - e^j,$$

$$w_j = \frac{d^j}{\sum_{j=1}^4 d^j},$$

其中， w_j 为得到的第 j 个指标的权重，将用于后续综合得分的计算，求解得到的权重如表 4 所示。

表 4 资源指数的各指标权重

指标	权重	指标	权重
医疗设施密度	0.1993	绿地密度	0.3212
养老机构密度	0.2609	文化设施密度	0.2186

4. 代入数据得到资源指数

$$R_i = \sum_{j=1}^4 w_j x_i^j。$$

根据上述四个指标的权重，代入其具体数据进行计算，可以得到第 x 个区域的资源指数，具体数据如表 5 所示。

表 5 各区域的资源指数大小

区域编号	资源指数	区域编号	资源指数
R1	0.7938	R11	0.9896
R2	1.1041	R12	2.9043
R3	1.4212	R13	0.6610
R4	1.2822	R14	1.0530
R5	3.7490	R15	0.9574
R6	7.1494	R16	1.2785
R7	0.6517	R17	0.8075
R8	1.4605	R18	0.8020
R9	3.8042	R19	0.9943
R10	0.7202	R20	0.6284

5.3 健康指数的模型建立与求解——模糊综合评价法^[2]

5.3.1 健康指数模型的准备

1. 定义指标、评价集与量化标准

(1) 指标：平均寿命 y^1 、慢性病发病率 y^2 及健康满意度 y^3 共同构建健康指数。

(2) 定性指标编码：将健康满意度转化为数值 $\{4,3,2,1,0\}=\{\text{好,较好,一般,较差,差}\}$ 。

(3) 评语集： $V=\{v_1,v_2,v_3,v_4,v_5\}=\{\text{好,较好,一般,较差,差}\}$ 。

(4) 量化分数：将评语转化为数值， $S=[4,3,2,1,0]$ ，其中“好”为 4 分，分值越高表示综合表现越好。

2. 构建隶属度矩阵

(1) 指标一致化处理

慢性病患者率为极小型指标，而其他两个指标为极大型指标，对其进行极大型指标的转化，以达到指标一致化的目的，

$$y^4 = y_{\max}^3 - y^3。$$

(2) 梯度隶属度函数

针对极大型指标（包含转化后的患病率），定义梯度函数参数：

$$\mu(v_j | y_i) = \begin{cases} 0, & y < a \text{ 或 } y > d \\ \frac{y-a}{b-a}, & a \leq y \leq b \\ 1, & b \leq y \leq c \\ \frac{d-y}{d-c}, & c \leq y \leq d \end{cases}。$$

其中， a 表示隶属度从 0 开始增长的起点， b 表示隶属度达到 1 的区间起点， c 表示隶属度开始下降的区间终点， d 表示隶属度降为 0 的终点，具体参数值请查看附录。

5.3.2 健康指数模型的求解

1. 计算隶属度矩阵

对每个指标的每个数据点，计算其对各评语的隶属度，利用上述梯度隶属度函数，形成矩阵 Q_j 。

2. 各指标权重的选取——熵值法

利用转换后的患病率，以及指标平均寿命、满意度，利用熵值法求解权重，方法与求解资源指数各指标权重方法相同，故不再赘述，得到的具体权重如表 6 所示。

表 6 健康指数的各指标权重大小

指标	权重
平均寿命（岁）	0.2327
慢性病患率（%）	0.2514
健康满意度（1-10 分）	0.5160

3. 模糊合成与健康指数综合得分计算

$$Q = \sum_{j=1}^3 k_j \cdot Q_j。$$

其中， k_j 为第 j 个指标的权重， Q 为综合隶属度矩阵，将各指标的隶属度矩阵与权重相乘后累加。

4. 计算健康指数即综合得分

$$H_i = \sum_{z=1}^5 Q_i^z \cdot S_z。$$

其中， Q_i^z 为第 i 个区域对第 z 个评语的隶属度， S_z 表示评语集中第 z 个评语的量化分数。

根据上述求解，可以得到各个区域的健康指数，具体数据内容如表 7 所示。

表 7 各区域的健康指数大小

区域编号	健康指数	区域编号	健康指数
R1	0.67	R11	3.07
R2	2.78	R12	2.50
R3	1.53	R13	4.38
R4	2.73	R14	4.12
R5	3.43	R15	1.35
R6	3.02	R16	1.92
R7	2.97	R17	2.37
R8	2.82	R18	2.32
R9	2.61	R19	1.13
R10	2.54	R20	2.59

5.4 资源-健康相关性分析

为检验康养资源分布是否能有效提高居民健康,构建资源指数与健康指数之间的皮尔逊相关系数:

$$r = \frac{\sum_{i=1}^{20} (R_i - \bar{R})(Q_i - \bar{Q})}{\sqrt{\sum_{i=1}^{20} (R_i - \bar{R})^2} \sqrt{\sum_{i=1}^{20} (Q_i - \bar{Q})^2}},$$

其中, $\bar{R} = \frac{1}{20} \sum_{i=1}^{20} R_i$ 为资源指数平均得分, $\bar{Q} = \frac{1}{20} \sum_{i=1}^{20} Q_i$ 为健康指数平均得分。

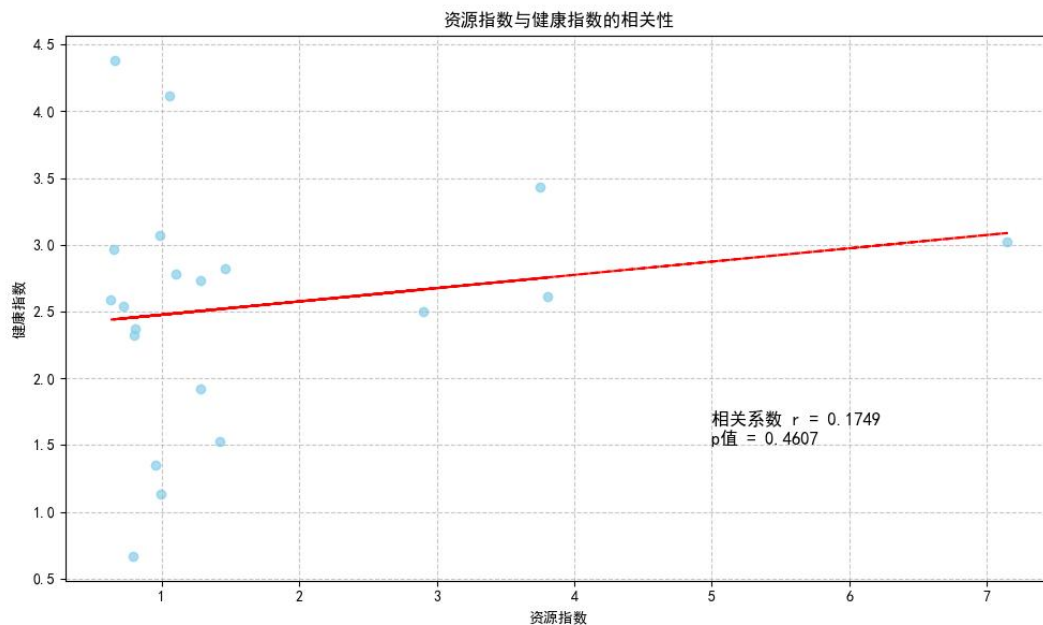


图 1 资源指数与健康指数的相关性

资源指数与健康指数的散点图及其相关性如图 1 所示，可得结果为 $r = 0.1749$ ，表明资源指数与健康指数之间为正相关关系，即资源指数越高，健康指数越高，符合常识。但总体而言皮尔逊相关系数显著偏离 1， p 值为 0.4607，表明资源指数与健康指数之间的相关性不显著，即存在资源配置不合理现象。

5.5 康养资源的目标优化模型建立于求解

为优化资源配置，解决资源分配不均的问题，以健康指数作为训练数据，构建资源指数关于总健康指数的多项式回归模型，在此基础上结合公平性评估技术，构建多目标优化模型，解决资源分配不均的问题。

5.5.1 目标函数的准备工作

1. 多项式回归模型的建立与求解

(1) 数据预处理

首先，过滤异常值，利用 Z-score 算法进行检验，通过数据点与均值的标准差倍数识别异常值，设数据服从正态分布，则其公式为：

$$Z - score = \frac{x_i - \mu}{\sigma},$$

其中， μ 为均值， σ 为标准差，数据点偏离均值超过 3 个标准差视为异常值。经检验，区域 6 为异常值，则剩余 19 个区域作为有效值保留，用于下述计算。

(2) 多项式回归模型的建立与求解

将总健康指数表示为资源指数 R_i 的多项式函数，建立健康指数与资源指数的关系：

$$T(R_i) = a_0 + a_1 R_i + a_2 R_i^2 + \cdots + a_n R_i^n,$$

其中，参数 a_i 通过最小二乘法拟合，结合有监督学习模型的训练集可得。

设拟合模型为 5 阶多项式，则计算可得总健康指数模型为：

$$T(\tilde{R}_i) = -1.5608 + 15.8397 \tilde{R}_i - 59.2341 \tilde{R}_i^2 + 101.4802 \tilde{R}_i^3 - 80.1268 \tilde{R}_i^4 + 25.8054 \tilde{R}_i^5$$

2. 公平性评估——基尼指数

为了量化和评估健康指数在区域间的分配公平性，实现“效率优先，兼顾公平”的资源分配目标，构建基尼指数，运用于后文中的资源目标优化模型。其具体计算公式为：

$$N = \frac{\sum_{i=1}^n (2i - n - 1) \cdot P_{(i)}}{n \cdot \sum_{i=1}^n P_{(i)}},$$

其中， $P_{(i)}$ 为排序后的健康指数， n 为区域数量，为 19 个。当 $N = 0$ 时，为绝对

公平；当 $0 < N < 0.2$ 时，为高度公平；当 $0.2 \leq N < 0.4$ 时，为中度公平；当 $N \geq 0.4$ 时，为不公平；当 $N = 1$ 时，为绝对不公平。

5.5.2 目标优化模型的建立

1. 目标函数的建立

在总资源固定的情况下，让所有区域的健康指数之和最大化，同时保证分配公平，以最大化总健康指数为目标函数，具体公式如下：

$$\min J = -\sum_{i=1}^{19} w_i T(\tilde{R}_i) + \rho \cdot N(T(\tilde{R}_1), T(\tilde{R}_2), \dots, T(\tilde{R}_{19})),$$

其中， ρ 是公平性权重，公平性权重越大，资源分配更平均；越小，效率权重越高，即资源倾向于高回报区域。

2. 约束条件的建立

(1) 所有区域资源之和等于总资源量

$$\sum_{i=1}^n \tilde{R}_i = \sum_{i=1}^n R_i$$

(2) 每个区域至少获得一定资源，避免极端不公平

$$\tilde{R}_i \geq 0.5, \quad i = 1, 2, \dots, 19$$

综上，以最大化总健康指数为目标函数，以资源总量不发生改变、每个区域至少获得一定资源为约束条件，构建目标优化模型为

$$\begin{aligned} \min J = & -\sum_{i=1}^{19} w_i T(\tilde{R}_i) + \rho \cdot N(T(\tilde{R}_1), T(\tilde{R}_2), \dots, T(\tilde{R}_{19})) \\ & \begin{cases} \sum_{i=1}^n \tilde{R}_i = \sum_{i=1}^n R_i \\ \tilde{R}_i \geq 0.5, \quad i = 1, 2, \dots, 19 \end{cases} \end{aligned}$$

5.5.3 目标优化模型的求解

多起点优化是一种全局优化策略，通过从多个不同的初始点启动局部优化算法，克服局部最优解的局限性，提高找到全局最优解的概率。序列最小二乘二次规划是专门设计用于求解带约束非线性优化问题的算法。已知上文中建立非线性目标函数，运用该算法并结合多起点最优算法进行求解，求解过程如下所示。

1. 随机生成初始点集合

$$c_1^{(0)}, c_2^{(0)}, \dots, c_N^{(0)}, \quad \text{其中 } c_k^{(0)} \sim \mu(\Omega)。$$

其中， Ω 为可行域，即满足所有约束条件的解空间， $\mu(\Omega)$ 表示在可行域内的均匀分布。在此基础上使用归一化可确保初始点满足总量约束。

2. 初始点局部最优

$$c_k^{(0)} = \min_{x \in \Omega} J。$$

其中， Ω 为满足所有约束条件的可行域， J 为目标函数。在具体计算中，使用序列最小二乘二次规划迭代求解，迭代公式为

$$c_k^{(t+1)} = c_k^{(t)} + \alpha_t \cdot d_k^{(t)},$$

$$\begin{cases} J(c^{(t+1)}) \leq J(c^{(t)}) + h_1 \alpha_t \nabla J(c^{(t)})^T d_k^{(t)} \\ |J(c^{(t+1)})^T d_k^{(t)}| \leq h_2 |J(c^{(t)})^T d_k^{(t)}| \end{cases}。$$

其中， $d_k^{(t)}$ 为二次规划子问题的搜索方向， α_t 为步长， h_1, h_2 约束为

$0 < h_1 < h_2 < 1$ 。通过迭代更新，使目标函数值单调下降，最终达到局部最优解。

3. 选择全局最优解

多起点优化通过多个初始点覆盖不同解空间区域，确保至少有一个起点的局部优化收敛或接近于全局最优。

综上，通过序列最小二乘二次规划算法搜索方向和步长的迭代调整，在满足约束的前提下，对每个初始资源分配方案进行局部优化，最终通过多起点共同确保求得全局最优解，优化资源配置。如图 2 所示，优化后总健康指数均有不同程度的提升，区域 1、区域 14 与区域 18 显著提升，表明居民健康得到显著优化。

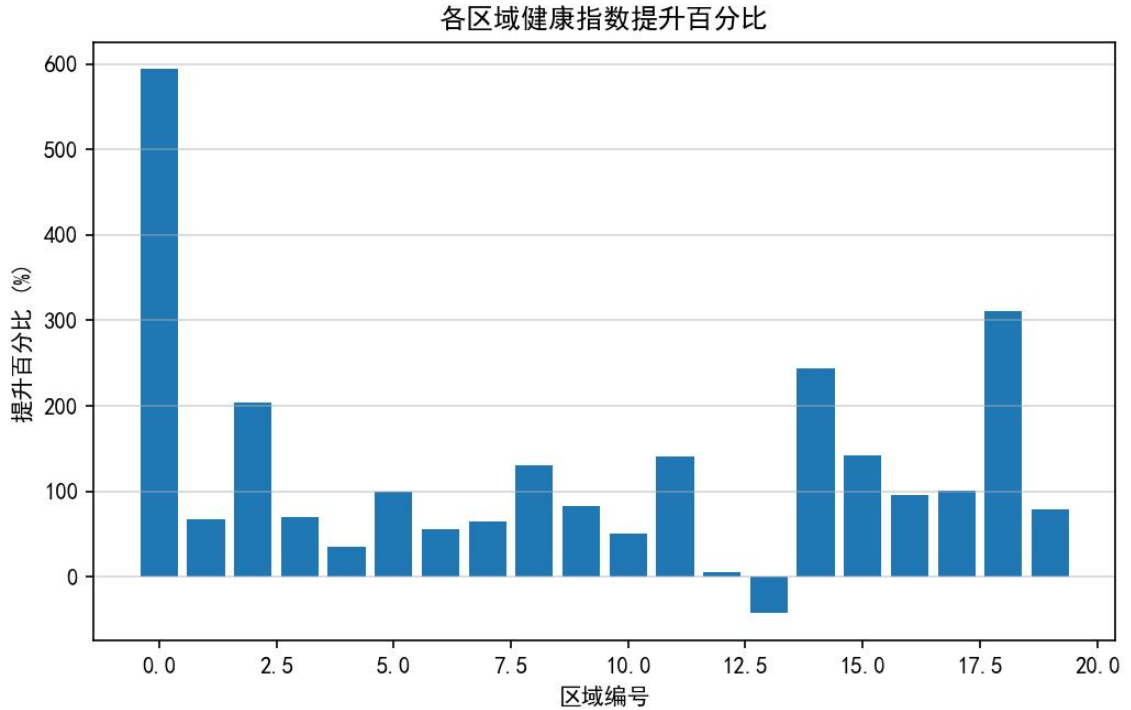


图 2 各区域健康指数提升百分比

由图 3、4 可见，针对资源分配与健康指数在优化前后的对比，可以看出在优化前资源分配极不均衡，部分区域的资源量高达 7，存在明显的资源分配不均

现象，优化后资源分配更加均匀，避免极端现象出现的同时，满足了最小资源约束，即每个区域都具有一定量的资源保证。健康指数在优化后普遍提升，整体总健康指数显著增加，优化较为合理。

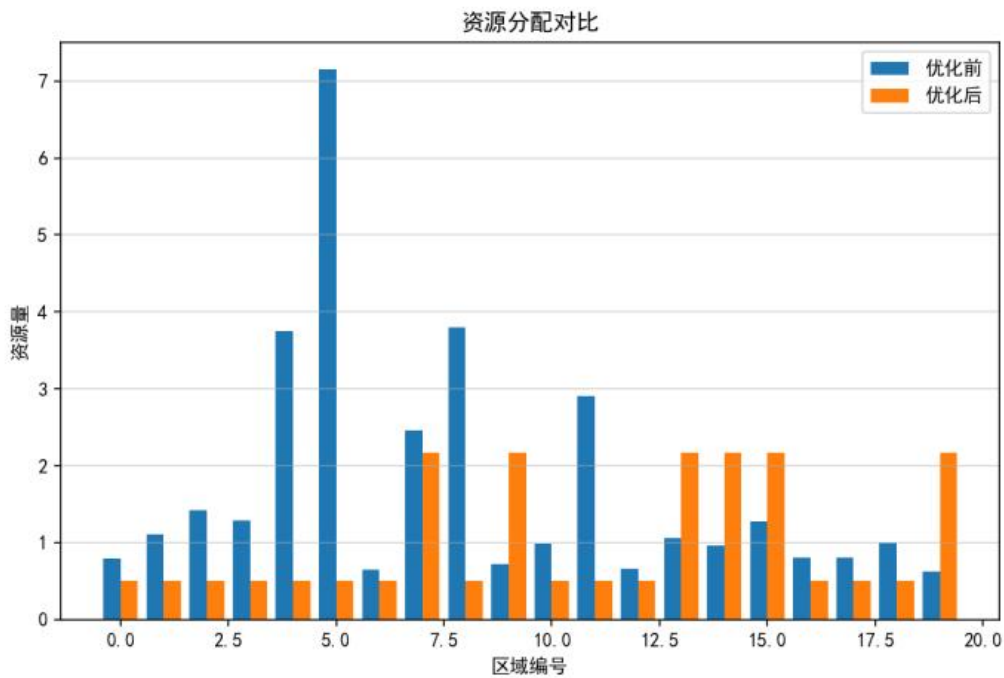


图 3 优化前后资源分配对比

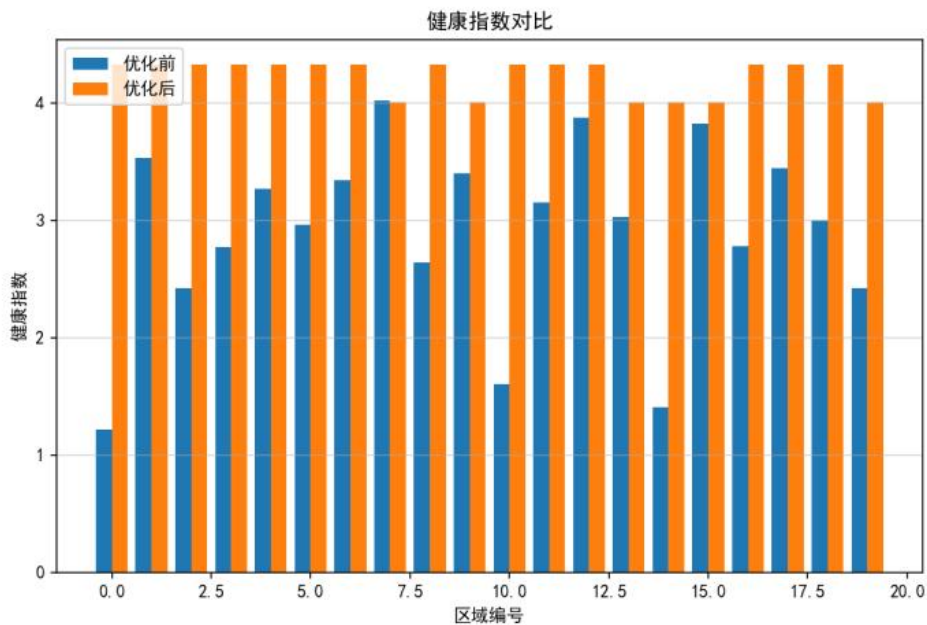


图 4 优化前后健康指数的对比

六、问题二模型的建立与求解

为确保评价的科学客观，本题采用熵权法与层次分析法结合确定指标权重，然后结合 TOPSIS 法计算综合得分排序，既保证了权重分配的客观性，又能直观

地给出各评价对象的优劣排序结果。并融合灰色关联分析（GRA）评估各城市与理想健康城市的贴近程度。

6.1 确定指标体系

为构建康养城市的综合评价模型，其评价不仅需要涵盖医疗、养老等传统康养资源指标，还需融合居民健康状态、生态环境和经济支撑能力等外部约束因素，因此本文将康养水平分解成一系列的可测量的具体维度。其中，一级维度为康养资源丰富度、居民健康状况、环境质量以及经济发展水平，二级维度（即一级维度的指标）为医疗设施密度、平均寿命、空气质量指数和人均 GDP 等详见表 6.1.1。

运用问题一的模型，根据各指标的详细数据，见支撑材料指标数据.xlsx，得到康养资源丰富度、居民健康状况、环境质量情况以及经济水平指数，汇总为表格，数据详见支撑材料维度指标.xlsx。各指标在一级维度下的权重，见表 6.1.1。

表 6.1.1 指标体系及各指标权重

一级维度	指标	权重
康养资源丰富度	x_1 :医疗设施密度	0.1993
	x_2 :养老机构密度	0.2609
	x_3 :公共绿地人均面积	0.3212
	x_4 :文化设施面积	0.2186
居民健康状况	x_5 :平均寿命	0.3504
	x_6 :慢性病发病率	0.3530
	x_7 :健康满意度	0.2186
环境质量	x_8 :空气质量指数	0.3232
	x_9 :水质达标率	0.3604
	x_{10} :噪音达标率	0.3164
经济水平	x_{11} :人均 GDP	1

6.2 综合评价模型的建立

为构建一个基于表 6.1 指标体系的多指标综合评价模型，即需要考虑数据本身的可观性差异，也要融合主管在政策权重。我们采用熵权法与层次分析法（AHP）

相结合的权重融合策略结合 TOPSIS 计算评分。

6.2.1 熵权法确定指标权重

原始数据的单位和量纲不同，如“寿命”（年）与“绿地面积”（平方米），不能直接加权求和。由于各指标量纲不同，对数据 x_{ij} 进行线性归一化处理，消除单位差异带来的影响。通过标准化，把所有指标变换到统一区间 $[0,1]$ ，以便比较、加权处理。

正向指标：

$$x_{ij'} = \frac{x_{ij} - \min_i x_{ij}}{\max_i x_{ij} - \min_i x_{ij}}。$$

负向指标：

$$x_{ij'} = \frac{\max_i x_{ij} - x_{ij}}{\max_i x_{ij} - \min_i x_{ij}}。$$

记标准化后第 j 项指标下第 i 个城市的值为 $x_{ij'}$ ，先求其在该指标下的占比

$$y_{ij} = \frac{x_{ij'}}{\sum_{i=1}^m x_{ij'}},$$

其中 m 为城市数量。然后计算第 j 项指标的信息熵：

$$e_j = -K \sum_{i=1}^m y_{ij} \ln y_{ij},$$

其中常数 $K = \frac{1}{\ln(m)}$ 。信息熵 e_j 介于 0 和 1 之间， e_j 越大表示该指标值分布越均匀、差异越小，提供的信息越少； e_j 越小说明该指标差异显著，提供信息量越大。

定义指标 j 的信息效用值为 $d_j = 1 - e_j$ 。 d_j 反映了指标 j 相对于“完全无区分信息”的差异程度，差异越大（即熵越小）， d_j 越大。最后计算第 j 项指标的熵权 w_j ：

$$w_j = \frac{d_j}{\sum_{j=1}^n d_j},$$

其中 $\sum_{j=1}^n w_j = 1$ 。如此得到各指标客观权重构成的权重向量 $\mathbf{w} = (w_1, \dots, w_n)$ 。

用 python 得结果见表 6.2.1。

表 6.2.1 熵权法结果

维度	熵值 e_j	信息效用 $d_j = 1 - e_j$	熵权 $w_j = \frac{d_j}{\sum d_j}$
康养资源丰富度	0.9714	0.0286	0.1523
居民健康状况	0.9679	0.0321	0.1710
环境质量	0.9903	0.0097	0.0516
经济发展水平	0.8828	0.1172	0.6251

6.2.2 AHP^[3]主观权重的计算

现实中决策者往往对某些维度有明确偏好：层次分析法可融入管理者意志，反映实际治理倾向，尤其在政策类问题中尤为重要。

本文采用的专家对于维度重要性的重要程度为：“资源≥健康≥环境≥经济”基于专家判断给出的各维度之间的相对重要性，采用 Saaty1-9 标度法构建 4×4 成对比较矩阵 $A=[a_{ij}]$ ，如下，

$$A = \begin{pmatrix} 1 & 3 & 5 & 7 \\ 1/3 & 1 & 3 & 5 \\ 1/5 & 1/3 & 1 & 3 \\ 1/7 & 1/5 & 1/3 & 1 \end{pmatrix}。$$

注:常用标度为 1 - 9 标度法（Saaty 标度），例如：“同等重要”取 1；“略微重要”取 3；“比较重要”取 5；“非常重要”取 7；“极端重要”取 9；中间可取偶数或倒数来表示偏好方向。

计算判断矩阵 A 的最大特征值 $\lambda_{\max}$ 及对应的特征向量 w^{AHP} ：

$$Aw^{AHP} = \lambda_{\max} w^{AHP}。$$

将特征向量归一化（和为 1）即得主观权重：

$$w^{AHP} \approx (0.5579, 0.2633, 0.1219, 0.0569)。$$

计算一致性检验指标 CI 与一致性比例 CR ，

$$CI = \frac{\lambda_{\max} - n}{n - 1}, \quad CR = \frac{CI}{RI},$$

其中 $n = 4$ ， RI 为经验随机一致性指标（ $n = 4$ 时 $RI = 0.90$ ）。若 $CR < 0.1$ ，则矩阵一致性可接受，否则需调整判断矩阵。通过 python 计算得：

$$\lambda_{\max} \approx 4.117，$$

$$CI = \frac{4.117 - 4}{4 - 1} \approx 0.039 ,$$

$$CR = \frac{0.039}{0.90} \approx 0.043 < 0.10 ,$$

符合一致性要求，可接受。

6.2.3 加权组合

设熵权法得客观权重为 w^E ，AHP 得主观权重为 w^{AHP} ，定义组合权重：

$$w = \alpha w^E + (1 - \alpha) w^{AHP} , \text{ 令 } \alpha = 0.5 ,$$

最后再归一化使 $\sum_j w_j = 1$ ，使用 python 计算归一化结果见表 6.2.2。

表 6.2.2 各指标权重

指标	组合权重 w
康养资源丰富度	0.3551
居民健康状况	0.2172
环境质量	0.0868
经济发展水平	0.3410

6.2.4 TOPSIS[4] 综合得分计算

TOPSIS 法依据贴近理想解程度对评价对象进行排序。基本原理是：假设存在一个“理想城市”，其各项指标取值都是所有城市中最优（正理想解），相应地还有一个各指标均最差的“负理想解”。通过计算每个待评城市与正理想解和负理想解的距离，比较谁“离理想最近、离最差最远”，即可评价各城市的优劣。

令有 $i = 100$ 个评价对象（城市）、 $j = 4$ 项评价指标，原始数据矩阵为

$$X = [x_{ij}]$$

将得到的组合权重应用于指标标准化，将 X 与权重相乘得到加权标准化指标矩阵。

$$z_{ij} = w_j \cdot x_{ij} ,$$

$$Z = [z_{ij}]_{m \times n} .$$

从矩阵 Z 中提取各指标的最大值组成正理想解向量

$$z_j^+ = \max_{1 \leq i \leq m} v_{ij} , \quad Z^+ = (z_1^+, \dots, z_n^+) ,$$

最小值组成负理想解向量

$$z_j^- = \min_{1 \leq i \leq m} v_{ij} \quad Z^- = (z_1^-, \dots, z_n^-) .$$

由于已正向化处理， z_j^+ 为指标 j 对各城市中最大值， z_j^- 为最小值。

采用加权欧氏距离公式，计算第 i 个城市与正理想解和负理想解的距离：

$$D_i^+ = \sqrt{\sum_{j=1}^n w_j (z_{ij} - z_j^+)^2},$$

$$D_i^- = \sqrt{\sum_{j=1}^n w_j (z_{ij} - z_j^-)^2}。$$

其中 D_i^+ 越小表示城市 i 越接近理想状态， D_i^- 越小表示越接近最差状态。

计算城市 i 的相对贴近度（得分）：

$$C_i = \frac{D_i^-}{D_i^+ + D_i^-}。$$

C_i 取值介于 0 和 1 之间，越接近 1 说明城市 i 距离理想解越近、综合评价越优。根据 C_i 大小对城市进行降序排序，即得到康养水平的排名，详见支撑材料 TOPSIS 得分.xlsx。排名前 10 见表 6.2.3。

表 6.2.3 TOPSIS 得分前 10

名次	城市	TOPSIS 得分
1	上海市	0.93331042
2	北京市	0.907890193
3	深圳市	0.814307127
4	广州市	0.695295871
5	重庆市	0.68676816
6	苏州市	0.537576234
7	成都市	0.469419139
8	杭州市	0.447522737
9	武汉市	0.52841433
10	南京市	0.406879191

6.3 灰色关联分析^[5]及建议

为分析各城市康养水平的优势与不足，分析各城市与“理想康养城市”的差距进行灰色关联分析以便提出针对性的意见。

6.3.1 灰色关联分析

首先，构建一个理想城市，记为：

$$X_0 = [x_0(1), x_0(2), \dots, x_0(n)],$$

取每个指标的最优值，令归一化后各指标均为 1，即 $x^0(j)=1(j=1,\dots,n)$ 。第 i 个城市记为：

$$X_i=[x_i(1),x_i(2),\dots,x_i(n)],x_i(j)=x'_{ij}。$$

计算理想城市与第 i 城市在第 j 个指标上的绝对差：

$$\Delta_i(j)=x_0(j)-x_i(j)。$$

将所有城市、所有指标的差值汇总，记

$$\Delta_{min}=\min_{i,j}\Delta_i(j)，\Delta_{max}=\max_{i,j}\Delta_i(j)。$$

对第 i 个城市第 j 个指标，定义灰色关联系数：

$$\gamma_i(j)=\frac{\Delta_{min}+\rho\Delta_{max}}{\Delta_i(j)+\rho\Delta_{max}}，\rho\in[0,1]$$

其中， ρ 为分辨系数（通常取 0.5）；当 $\Delta_i(j)$ 越小， $\gamma_i(j)$ 越接近 1，说明城市 i 在指标 j 上越接近理想水平。

将各指标关联系数按权重 $\{w_j\}_{j=1}^4$ 加权平均，得到第 i 城市的灰色综合关联度：

$$\Gamma_i=\sum_{j=1}^nw_j\gamma_i(j)，\sum_{j=1}^nw_j=1。$$

其中， $\Gamma_i\in[0,1]$ ，值越大说明城市 i 的整体发展模式越接近“理想健康城市”。

按 Γ_i 从大到小对 m 个城市进行排序，可得到康养水平的分层序列。排名前 10 见表 6.3.1。优势分析：关联度高的城市，其优势指标往往在少数维度（如环境或健康）表现突出；不足分析：关联度低的城市，与理想城市的差异主要集中在某些指标上，可针对这些指标制定提升策略。

表 6.3.1 灰色关联度前 10

排名	城市	康养指数	居民健康状况	环境质量	经济发展水平	灰色关联度
1	深圳市	0.76	0.97	0.42	0.73	0.77
2	上海市	0.53	0.86	0.44	1.00	0.76
3	北京市	0.53	0.92	0.53	0.88	0.73
4	杭州市	0.85	1.00	0.49	0.47	0.72
5	广州市	0.99	0.50	0.49	0.62	0.71
6	苏州市	0.65	0.94	0.47	0.52	0.65
7	南京市	0.76	0.85	0.46	0.45	0.65

8	重庆市	0.81	0.46	0.53	0.61	0.64
9	成都市	0.75	0.67	0.53	0.48	0.62
10	海口市	1.00	0.50	0.38	0.34	0.61

6.3.2 结果分析及合理性建议

根据灰色关联分析结果，针对不同类型城市的评价结果，可提出如下分类型康养建设策略：

I类（经济引领型）城市：代表如上海、广州等综合得分领先的大城市。它们拥有充裕的医疗康养资源和经济实力，居民健康指标也处于前列。短板主要在环境与生活品质方面。建议此类城市以提升环境质量为抓手，严格控制大气污染和城市噪音，增加城市绿地和康养休闲空间，以匹配其高水平的医疗服务供给。同时，鼓励利用经济和科技优势，开发智慧康养服务，辐射带动周边地区。

II类（生态康养型）城市：代表如海口、三亚等环境宜居的旅游城市。它们空气洁净、气候舒适，是天然的康养胜地，但在高级医疗资源和产业支撑方面相对薄弱。建议此类城市发挥生态环境优势，定位为全国康养旅游目的地，引进高水平医疗康复机构和专业人才。通过政策扶持“候鸟式”养老产业、建设康养小镇等方式，将绿水青山转化为康养产业发展的金山银山。同时，加大教育和招商力度，逐步提升本地经济发展水平，为康养产业提供更坚实的基础。

III类（均衡发展型）城市：代表如杭州、重庆等各项指标较为均衡的城市。它们经济较强、医疗资源丰富，环境和健康状况中等偏上，但没有特别突出的单项优势。建议此类城市对标更高标准，在保持均衡发展的同时寻求突破口。例如杭州可进一步改善大气环境和水体治理，打造“宜居宜养”的城市品牌；重庆可结合山地气候和中医药资源，发展特色康养疗法，形成自身亮点。

IV类（基础提升型）城市：代表如梧州等欠发达中小城市。它们在经济、资源、健康等各方面与理想水平差距较大。对此应加强政府主导作用，夯实基础设施和公共服务：加大医疗卫生投入，提升基层医疗和养老院所覆盖面；推动产业转型升级，改善就业和收入水平；加强环境污染治理和人居环境整治，防止健康危害。通过补短板促发展，逐步提升居民健康福祉。

总之，不同城市应根据自身康养发展评价结果，有针对性地制定健康城市建设方案，扬长避短。对于优势领域，政府应继续投入和支持，巩固提升竞争力；对于薄弱环节，要列入重点整改清单，制定阶段性目标不断改善。通过分类施策、协同推进，最终实现各类城市健康、养老、环境、经济的协调发展，全面提升城市康养水平，为居民创造高质量的健康宜居生活环境。

七、问题三模型的建立与求解

康养城市建设的核心在于通过科学配置医疗、养老等资源，实现“老有所养、病有所医”的民生目标。然而，当前资源配置面临公平性、积极性、效率性三大矛盾。因此，本研究通过构建多目标函数优化模型，为政府决策提供精准工具。主要过程如图 7.1 的流程图。

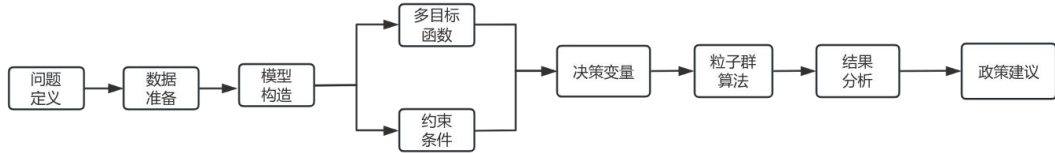


图 7.1 问题三流程图

7.1 优化模型的建立

7.1.1 确定决策变量

1. 核心离散变量（设施选址与流动车部署）

针对设施选址变量，康养设施选址需兼顾人口需求与建设成本，同时 0-1 变量直接对应现实中“建或不建”的决策，因此在优化模型中将康养设施的选址作为决策变量，即

$$x_i = \begin{cases} 1, & \text{在第 } i \text{ 个候选位置建设康养设施} \\ 0, & \text{不在第 } i \text{ 个候选位置建设} \end{cases}, \quad i = 1, 2, \dots, m$$

其中， m 为候选设施总数。

针对流动车部署变量，考虑到偏远地区无法建设固定设施时，通过流动车补充覆盖，同时 0-1 变量直接对应现实中“部署与不部署”的决策，因此在优化模型中将流动车步数作为决策变量，即

$$z_k = \begin{cases} 1, & \text{部署第 } k \text{ 辆流动服务车} \\ 0, & \text{不部署第 } k \text{ 辆流动服务车} \end{cases}, \quad k = 1, 2, \dots, p$$

其中， p 为流动车的总数量。

2. 连续性服务变量（服务占比及时段分配）

针对服务量分配变量，为解决主城区单设施容量不足问题，同时落实“医养结合”政策中多机构协同服务要求，模型采用服务量占比变量作为决策变量，即

$$y_{ij} \in [0, 1], \quad i = 1, 2, \dots, m; j = 1, 2, \dots, n$$

其中， n 为需求区域总数， y_{ij} 表示设施 i 为区域 j 提供的服务量占需求区域 j 总需求的比重。

针对分时服务量分配变量，考虑到老年人对昼夜服务差异化的要求，为实现分时段资源调度，模型引入分时服务量作为决策变量，即

$$y_{ijt} \in [0,1], t=1,2$$

其中， $t=1$ 为日间， $t=2$ 为夜间

针对设施规模变量，考虑到设施容量和服务人数的比例，为确保设施规模匹配需求，防止设施过载。因此将设施规模变量 s_i 作为决策变量。

7.1.2 确定目标函数

1. 针对康养服务覆盖率

首先，为“落实优先保障高龄、高健康风险群体服务需求”的政策要求，模型通过加权因子 w_j 量化区域需求优先级，实现资源差异化分配。计算公式如下：

$$w_j = 0.5\rho_j^{old} + 0.3\rho_j^{chd} + 0.2\rho_j^{sol}$$

其中，老龄化 (ρ_j^{old}) 权重最高，响应对高龄群体的优先保障；慢性病率 (ρ_j^{chd}) 与独居率 (ρ_j^{sol}) 反应健康风险与照料需求，权重分别为 0.3 和 0.2。

然后，为确保流动车应急响应时效性，模型引入借助示性函数来精准量化流动车对区域的服务覆盖能力，具体定义如下：

$$I(d_{kj} \leq r_k^{流动}) = \begin{cases} 1, & \text{当流动车 } k \text{ 到区域 } j \text{ 的距离 } d_{kj} \leq \text{覆盖半径 } r_k^{流动} \\ 0, & \text{否则} \end{cases}$$

其中， d_{kj} 代表流动车 k 的部署位置到需求区域 j 的实际空间距离， $r_k^{流动}$ 表示流动车 k 的有效覆盖半径，依据政策标准，农村地区通常设定为 5 公里，以保障应急响应时效性。

最后，应该要确保每个区域 j 在时段 t 的需求必须被固定设施或距离达标的流动车完全覆盖，即

$$\sum_{i=1}^m y_{ij} + \sum_{k=1}^p z_k \cdot I(d_{kj} \leq r_k^{流动}) = 1$$

综上，根据题目要求建立一种康养资源优化配置模型，因此选择康养服务覆

盖率最大化作为目标函数，即

$$\max f_1 = \sum_{j=1}^n w_j \cdot \left(\sum_{i=1}^m y_{ij} + \sum_{k=1}^p z_k \cdot I(d_{kj} \leq r_k^{\text{流动}}) \right)$$

其相对应的约束条件为：

$$\begin{cases} w_j = 0.5\rho_j^{\text{old}} + 0.3\rho_j^{\text{chd}} + 0.2\rho_j^{\text{sol}} \\ \sum_{i=1}^m y_{ij} + \sum_{k=1}^p z_k \cdot I(d_{kj} \leq r_k^{\text{流动}}) = 1 \\ I(d_{kj} \leq r_k^{\text{流动}}) = \begin{cases} 1, & \text{当流动车 } k \text{ 到区域 } j \text{ 的距离 } d_{kj} \leq \text{覆盖半径 } r_k^{\text{流动}} \\ 0, & \text{否则} \end{cases} \end{cases}$$

其中， y_{ij} 表示设施 i 为区域 j 提供的服务量占需求区域 j 总需求的比重， d_{kj} 代表流动车 k 的部署位置到需求区域 j 的实际空间距离， $r_k^{\text{流动}}$ 表示流动车 k 的有效覆盖半径， w_j 代表加权因子。

2. 针对最小化成本投入

根据题目要求建立一种康养资源优化配置模型，因此选择成本投入最小化作为目标函数，来实现财政压力与服务供给的平衡，即

$$\min f_2 = \underbrace{\sum_{i=1}^m (c_i^{\text{建设}} s_i + c_i^{\text{运营}} s_i T) x_i}_{\text{固定设施成本}} + \underbrace{\sum_{k=1}^p c_k^{\text{流动}} z_k}_{\text{流动车成本}} + \underbrace{\sum_{i,j,t} d_{ijt} \cdot y_{ijt} \cdot c^{\text{配送}}}_{\text{分时配送成本}}$$

其中， $c_i^{\text{建设}}$ 表示第 i 个固定设施的单位建设成本，这里假设候选点建设成本为 800 万元/千平方米； $c_i^{\text{运营}}$ 表示第 i 个固定设施的单位运营成本，假设按建设成本的 10%/年来计算，所以候选点运营成本为 80 万元/年； $c_k^{\text{流动}}$ 表示第 k 辆流动车的年平均成本，取值约为 50 万元； $c^{\text{配送}}$ 表示配送成本系数，日间取值为 1，夜间取值为 1.2； d_{ijt} 表示在 t 时段固定设施 i 到区域 j 的配送距离。

3. 针对资源利用率

为避免出现康养设施闲置率高的问题，通过建立资源利用率函数来量化设施服务效率，推动资源高效配置，计算公式如下：

$$\max f_3 = \sum_{i=1}^m \frac{\sum_{j,t} y_{ijt} \cdot d_j}{s_i \cdot T}$$

其中, $\sum_{j,t} y_{ijt} \cdot d_j$ 代表设施 i 在所有时段 t 内, 为各区域 j 提供的实际服务量总和, $s_i \cdot T$ 代表在 T 年内该设施的最大服务量。

7.1.3 确定约束条件

1. 针对容量约束

因为消防安全规范对夜间人员密度有限制要求, 且实际运营中需预留应急容量以应对突发需求, 同时避免设施超负荷影响服务质量, 所以设置约束条件

$$\sum_{j=1}^n y_{ijt} \cdot d_j \leq s_i \cdot \eta_t$$

其中, $\sum_{j,t} y_{ijt} \cdot d_j$ 代表设施 i 在所有时段 t 内, 为各区域 j 提供的实际服务量总和, η_t 为时段容量数, 日间 $\eta_1 = 0.8$ 为常规运营负荷, 夜间 $\eta_2 = 0.5$ 为应急模式。

2. 针对预算约束

考虑到地方财政资金存在总量限制, 所以设置约束控制康养资源配置总支出不超过财政预算上限, 即

$$\sum_{i=1}^m c_i^{\text{建设}} s_i x_i + \sum_{k=1}^p c_k^{\text{流动}} z_k \leq c$$

$$\sum_{i=1}^m c_i^{\text{建设}} s_i x_i + \sum_{k=1}^p c_k^{\text{流动}} z_k \leq c$$

其中, c 为财政预算, $c_i^{\text{建设}}$ 表示第 i 个固定设施的单位建设成本, $c_i^{\text{运营}}$ 表示第 i 个固定设施的单位运营成本。

4. 设置关联性约束

由于未建设的设施无法提供服务, 所以设置约束条件

$$y_{ijt} \leq x_i$$

7.1.4 多目标优化模型的建立

选择最大化康养服务覆盖率、最小化成本投入和最大化资源利用率为目标函数, 设施选址变量 x_i 、流动车部署变量 z_k 、服务量分配变量 y_{ij} 等作为优化变量,

容量约束、预算约束、服务覆盖等为约束条件，建立优化模型：

$$\begin{aligned}
\max f_1 &= \sum_{j=1}^n w_j \cdot \left(\sum_{i=1}^m y_{ij} + \sum_{k=1}^p z_k \cdot I(d_{kj} \leq r_k^{\text{流动}}) \right) \\
\min f_2 &= \underbrace{\sum_{i=1}^m (c_i^{\text{建设}} s_i + c_i^{\text{运营}} s_i T) x_i}_{\text{固定设施成本}} + \underbrace{\sum_{k=1}^p c_k^{\text{流动}} z_k}_{\text{流动车成本}} + \underbrace{\sum_{i,j,t} d_{ijt} \cdot y_{ijt} \cdot c^{\text{配送}}}_{\text{分时配送成本}} \\
\max f_3 &= \sum_{i=1}^m \frac{\sum_{j,t} y_{ijt} \cdot d_j}{s_i \cdot T} \\
\begin{cases} w_j = 0.5 \rho_j^{\text{old}} + 0.3 \rho_j^{\text{chd}} + 0.2 \rho_j^{\text{sol}} \\ \sum_{i=1}^m y_{ij} + \sum_{k=1}^p z_k \cdot I(d_{kj} \leq r_k^{\text{流动}}) = 1 \\ \sum_{j=1}^n y_{ijt} \cdot d_j \leq s_i \cdot \eta_t \\ \sum_{i=1}^m c_i^{\text{建设}} s_i x_i + \sum_{k=1}^p c_k^{\text{流动}} z_k \leq c \\ y_{ijt} \leq x_i \\ y_{ij} \in [0,1], \quad i=1,2,\dots,m; j=1,2,\dots,n \\ y_{ijt} \in [0,1], t=1,2 \\ I(d_{kj} \leq r_k^{\text{流动}}) = \begin{cases} 1, & \text{当流动车 } k \text{ 到区域 } j \text{ 的距离 } d_{kj} \leq \text{覆盖半径 } r_k^{\text{流动}} \\ 0, & \text{否则} \end{cases} \\ z_k = \begin{cases} 1, & \text{部署第 } k \text{ 辆流动服务车} \\ 0, & \text{不部署第 } k \text{ 辆流动服务车} \end{cases}, \quad k=1,2,\dots,p \\ x_i = \begin{cases} 1, & \text{在第 } i \text{ 个候选位置建设康养设施} \\ 0, & \text{不在第 } i \text{ 个候选位置建设} \end{cases}, \quad i=1,2,\dots,m \end{cases}
\end{aligned}$$

7.2 多目标优化模型的求解

粒子群优化算法是一种基于群体智能的全局优化算法，适用于求解康养资源优化配置这类多目标、多变量问题。以下是基于该模型的粒子群优化算法具体求解流程图，见图 7.2.1。

7.2.1 参数与粒子群^[6]初始化

1. 设定基础参数：设置粒子群规模 $N=100$ （即种群包含100个粒子）；最大迭代次数 $T_{\max}=200$ ；初始值 $w=0.9$ ，随迭代线性减至 $w=0.4$ ；调整粒子向个体最优和全局最优学习的步长，学习因子为 $c_1=c_2=1.5$ ，设置 f_1, f_2, f_3 三个目标函数

的权重为 0.4、0.3、0.3。

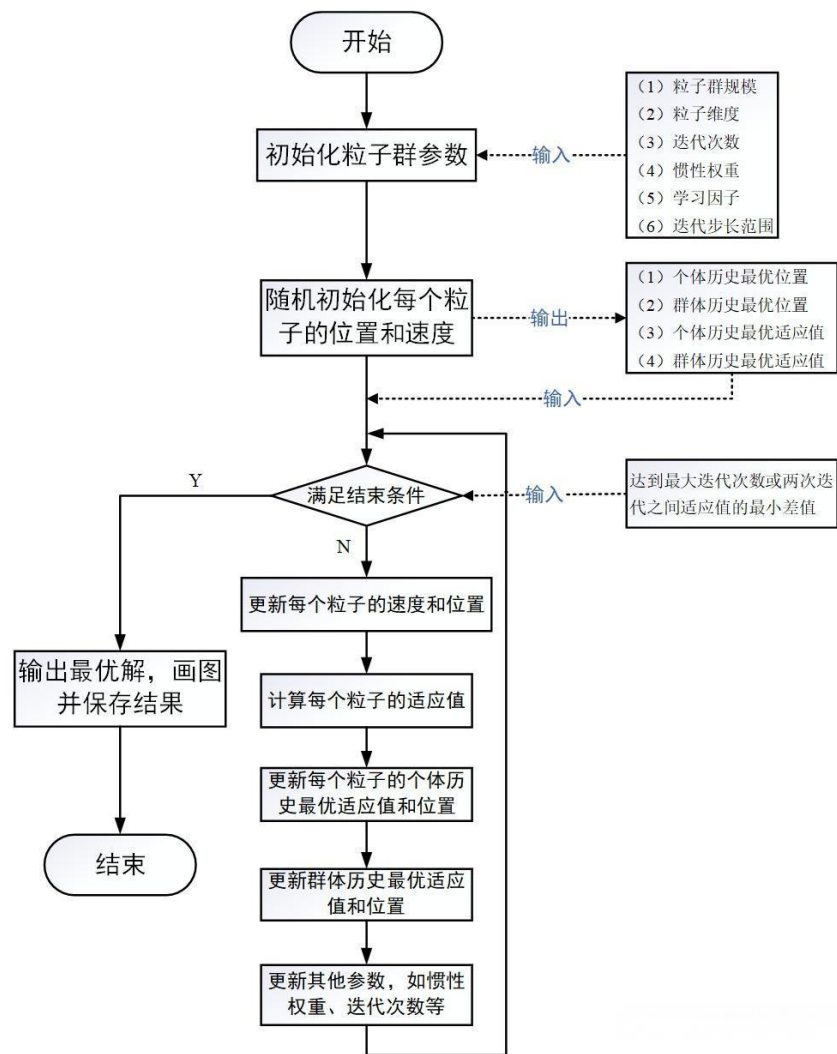


图 7.2.1 粒子群优化算法流程图

2. 粒子编码与随机生成

首先设置粒子群结构，每个粒子代表一个资源配置方案，编码为 $[X, Z, Y, B, S]$ ，其中 $X = [x_i]$ 、 $Z = [z_k]$ 、 $Y = [y_{ij}]$ 、 $B = [b_{ijt}]$ 、 $S = [s_i]$ ；然后初始化规则，离散变量 X 、 Z 随机取 0 或 1，连续变量随机 Y 、 B 、 S 在合法区间上随机生成

7.2.2 适应函数计算

首先，对于每个粒子，根据其变量取值计算三个目标函数的取值，然后将各个目标函数的取值归一化，最后进行适应度计算，同时若粒子违反约束条件，根据违规次数添加惩罚项，具体公式如下：

$$fit = \sum_{k=1}^3 w_k \cdot Norn(f_k) + penalty$$

其中， $penalty = \text{违规次数} \times 10^4$ 。

7.2.3 个体最优和全局最优更新

对于每个粒子，比较当前适应度和历史最优适应度，若当前适应度大于历史最优适应度则更新当前该粒子的个体最优解为当前解。遍历所有粒子，找出适应度最高的粒子，将其设置为全局最优解。

7.2.4 粒子速度与位移更新

首先，对每个粒子的每个维度按照以下公式更新速度：

$$V_k^{t+1} = w \cdot V_k^t + c_1 r_1 (pbest_k - X_k^t) + c_2 r_2 (gbest_k - X_k^t)$$

其中， $r_1, r_2 \sim U(0,1)$ ， w 随迭代次数线性递减： $w = 0.9 - t \times \frac{0.5}{T_{\max}}$ 。

然后，针对离散变量通过 sigmoid 函数将速度映射为概率，决定 0-1 的取值；针对连续变量，直接累加速度更新位置，并修正至合法区间。

最终，迭代次数到达 200，输出各个优化变量的值。

7.3 结果输出与分析

7.3.1 模拟数据

结合问题一的数据，考虑到优化模型需要空间距离，在 python 中通过调整随机种子或坐标范围，来生成候选点所在的位置，如下表所示

表 7.3.1 候选点所在位置

候选点编号	经度	维度
1	120.7862	31.0046
2	121.3953	30.9861
3	120.9941	31.0641
4	121.1093	31.2259
5	120.5943	30.8523
.....		
20	120.6727	31.2832

该候选点列表为康养设施选址提供了标准化、可复现的空间基础，通过地理覆盖与均匀分布，支撑后续的资源配置、服务优化等决策，是实现康养设施精准布局的关键数据输入。同时生成地理经纬度图，如下图所示。

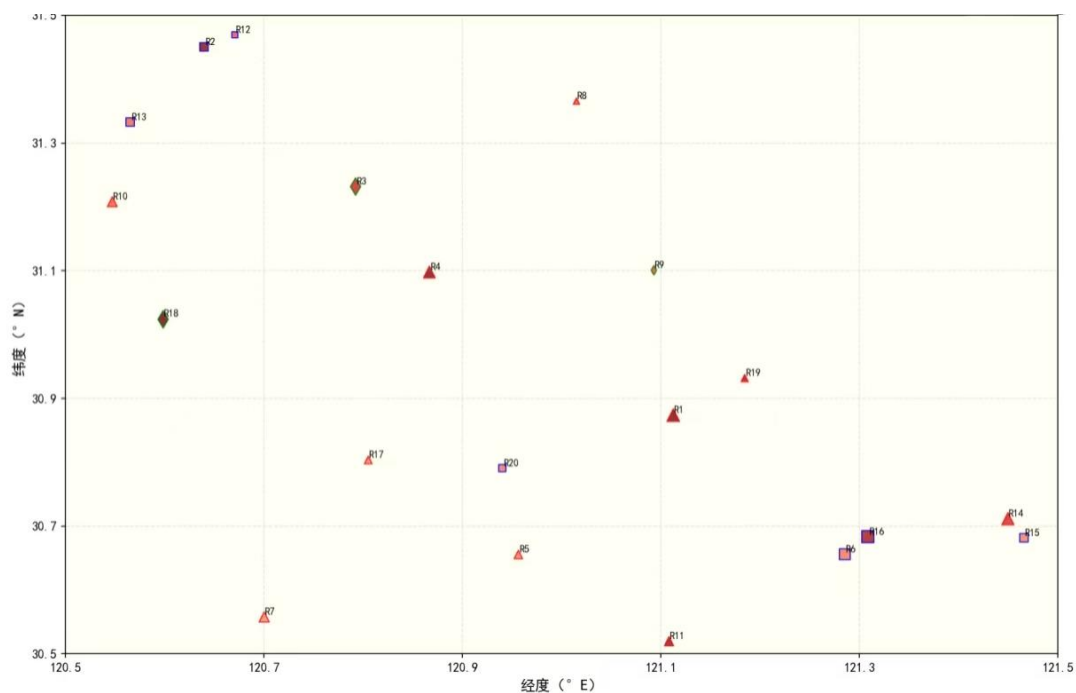


图 7.3.1 候选点地理位置

7.3.2 算法求解

结合问题一和利用 python 随机生成的城市区域数据，代入优化模型中并进行粒子群算法求解，经过 200 次迭代，最终得到最佳适应度为 0.3144。同时将优化结果进行空间可视化，如下图所示。

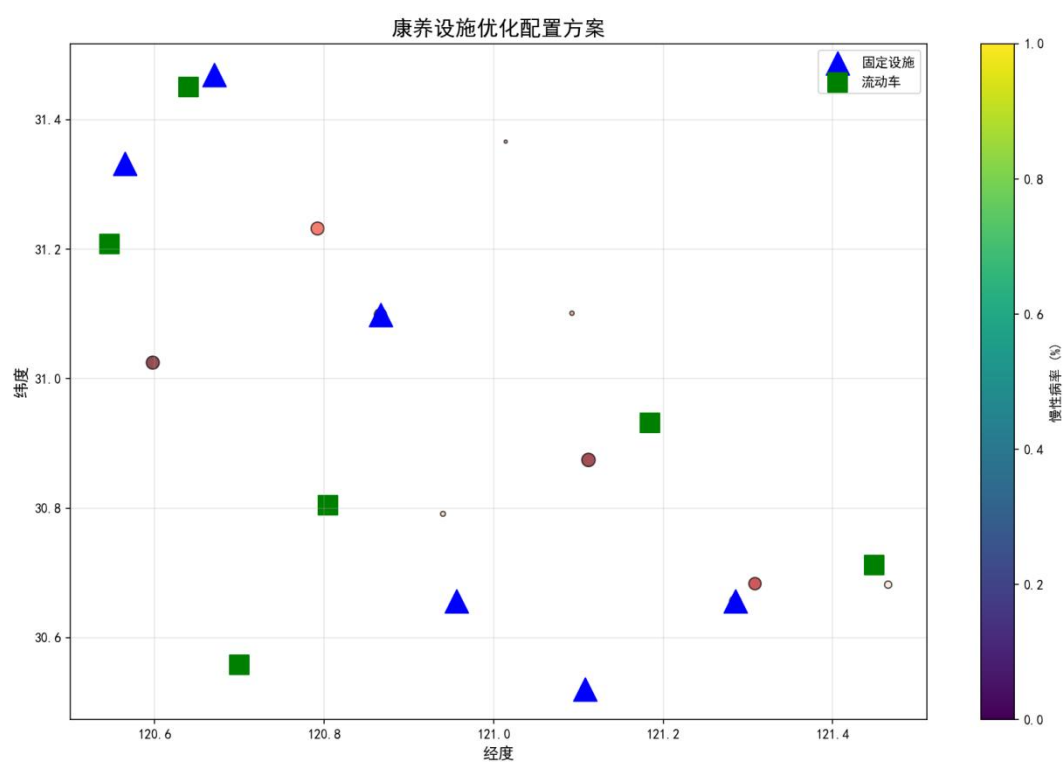


图 7.3.2 康养设施优化配置方案

该图直观呈现“核心区固定设施主导，边缘区流动车补充”的优化逻辑，实现资源精准投放与全域覆盖，为康养设施布局、服务调度提供科学决策支持，提升资源利用效率与服务可及性。

7.3.3 优化变量分析

1. 设施选址与规模决策

结合区域的具体位置和下表，可明显发现设施选在区域人口较密集且慢性病率高的区域，表明能满足大量患者要求，提升服务效率与效果。例如 R12 设施规模 180 最大，因为该区域人口多、慢性病率高，需容纳更大患者接受服务。

表 7.3.2 设施选址与规模变量决策

设施编号	区域	设施规模
1	R5	160
2	R12	180
3	R8	170
4	R18	165

2. 流动车部署决策

结合区域的具体位置和下表，可明显发现流动车部署在边缘或者次高需求区域提供灵活服务，弥补固定设施覆盖不足。例如 R3 虽人口多但地理位置较偏僻，流动车可定期巡回服务，提高服务的可确性。

表 7.3.3 流动车部署变量决策

流动车编号	部署区域
1	R3
2	R7
3	R15

3. 服务量分配决策

分析下表，可发现区域与设施对应，有本地设施的区域，优先将服务量分配给本地设施；同时按照距离导向分配，无本地设施区域按距离分配。例如 R5 固定设施向 R5 提供的服务量达到 62%，R14 大部分服务量分给临近的 R18 区域，达到优化资源利用，确保患者得到及时服务的效果。

表 7.3.4 流动车部署决策

需求区域	R5（固定）	R12（固定）	R8（固	R18（固	R3（流	R7（流动）	R15（流动）
------	--------	---------	------	-------	------	--------	---------

			定)	定)	动)		
R1	0.16	0.00	0.00	0.00	0.21	0.11	0.00
R2	0.00	0.13	0.00	0.00	0.00	0.16	0.09
R3	0.00	0.00	0.00	0.00	0.82	0.00	0.00
R4	0.00	0.00	0.27	0.00	0.00	0.32	0.16
R5	0.62	0.00	0.00	0.00	0.00	0.00	0.00
.....							
R20	0.00	0.00	0.00	0.53	0.00	0.00	0.11

同时通过对比白天与夜晚的服务量分配决策，可明显发现

（I）时段服务差异显著：白天和晚上服务量分配有明显区别，白天占全天服务量发起 80%，是主要的服务时段，晚上仅占 20%且多为紧急需求和基础护理，这表明资源在配置和安排上需重点保护白天时段的服务供给；

（II）区域需求不匹配：不同地区对康养服务的需求差异较大，高需求区域（如 R5、R12）在白天的服务分配比例远高于晚上，说明这些区域在白天的康养服务需求更为旺盛；而像 R3 这类流动车重点覆盖区域，白天和晚上的服务量占比变化也反映出其需求特点，流动车在白天承担了主要服务任务。

（III）设施服务有侧重：固定设施和流动车在不同区域和时段的服务分配各有侧重。固定设施（如 R18 对 R14）在白天对周边区域提供主要服务，晚上服务量减少；流动车（如 R3）在特定区域的服务量随时间变化明显，体现了其灵活调配的特点。

表 7.3.5 晚上时段服务量分配决策

需求区域	R5（固定）	R12（固定）	R8（固定）	R18（固定）	R3（流动）	R7（流动）	R15（流动）
R1	0.08	0.00	0.00	0.00	0.10	0.05	0.00
R2	0.00	0.06	0.00	0.00	0.00	0.07	0.04
R3	0.00	0.00	0.00	0.00	0.40	0.00	0.00
R4	0.00	0.00	0.13	0.00	0.00	0.15	0.07
R5	0.30	0.00	0.00	0.00	0.00	0.00	0.00
.....							
R20	0.00	0.00	0.00	0.25	0.00	0.00	0.05

4.关键优化指标

针对服务覆盖率：达 88.15%，表明当前设施与流动车布局及服务分配方案

能覆盖大部分区域需求，多数居民能获得康养服务，但仍有提升空间，可进一步优化资源布局扩大覆盖。

针对总成本：18.63 百万元，涵盖设施建设、运营、人员薪资等成本，是衡量资源投入的关键指标，可用于评估方案经济性，为成本控制和资源优化提供参考。

针对资源利用率：82.47%，反映设施床位、医护人员等资源实际使用程度，说明资源利用较充分，但仍有部分资源未被完全利用，可分析闲置原因进一步优化资源配置。

7.4 政策建议

针对设施布局：固定设施聚焦人口密、慢性病率高区域（如 R5、R12），流动车补位边缘区（如 R3、R15）。借助 GIS 动态规划选址，优化流动车路线，提升覆盖与效率，使资源与需求精准匹配，强化核心区服务能力，填补边缘区空白，构建全域康养网络。

针对服务分配：本地设施优先服务（如 R5→R5），无则按距离分配（如 R14→R18）。建立实时调度系统，依时段（白天强化高需求区，晚上保障核心区应急）、空间（细化距离权重）动态调服务量，实现资源精准投放，提升响应速度，满足居民多样化康养需求。

针对成本与资源管理：细化成本核算，分类管控建设、运营、人力成本，推行绿色运营降能耗。整合闲置床位、设备跨区共享，提升资源利用率。以绩效考评驱动资源优化，减少浪费，平衡成本与效益，确保康养服务在经济可持续下高效运行，最大化资源价值。

针对质量与监督：统一服务标准，开展专项培训（如慢性病管理、应急护理）提升医护水平。建立公众反馈机制，将覆盖率、利用率等纳入考核，整改未达标区域，确保核心与边缘区服务质量一致，增强居民满意度，筑牢康养服务质量防线，保障服务公平性与可靠性。

八、模型的评价与优化

8.1 模型评价

优点：多目标平衡公平性、效率与成本，变量体系完整覆盖选址、调度等决策，粒子群算法实现资源利用率 82.47%，结果结合 GIS 可视化，政策建议落地

性强。

缺点：目标权重主观性强，缺乏动态约束（如交通、人口流动），算法参数未优化，未提供多目标解集，数据依赖随机生成、可复现性不足。

8.1 优化建议

方法改进：用 AHP /德尔菲法科学设定权重，引入动态约束（如时间距离覆盖、人口预测），结合遗传算法提升优化效果，生成帕累托解集供决策选择。

数据与验证：接入 GIS、人口普查等真实数据，通过敏感性测试增强鲁棒性。

技术融合：集成 IoT 动态调度系统，开发可视化决策平台，提升模型实时响应与交互性。

模型为康养资源配置提供基础框架，优化后可更贴合现实需求，提升决策科学性。

参考文献

- [1] 王岩 .Monte Carlo 方法应用研究 [J]. 云南大学学报（自然科学版）,2006,28(S1):23-26.
- [2] 蔡天欣.智能养老服务质量评估机制研究——基于 AHP—模糊综合评价法[J]. 上海城市管理,2024,33(6):28-34.
- [3] 郭金玉,张忠彬,孙庆云.层次分析法的研究与应用[J].中国安全科学学报,2008,18(5):148-153.
- [4] 王瑛,常泉英.基于二次赋权的 TOPSIS 法的城市环境质量动态评价[J].安全与环境学报,2018,18(2):784-788.
- [5] 刘思峰,蔡华,杨英杰,曹颖.灰色关联分析模型研究进展[J].系统工程理论与实践,2013,33(8):2041-2046.
- [6] 陈绍新. 多目标优化的粒子群算法及其应用研究[D]. 辽宁:大连理工大学,2007. DOI:10.7666/d.y1226632.

附录

问题一某城市康养的模拟数据

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 设置随机种子以便复现实验结果
np.random.seed(42)

# 假设城市划分为 20 个区域
num_regions = 20

# 定义健康满意度的 5 个等级
satisfaction_levels = ['差', '较差', '一般', '较好', '好']

# 定义每个等级的概率
probabilities = [0.1, 0.2, 0.4, 0.2, 0.1]

# 模拟数据生成
data = {
    "区域编号": [f"R{i+1}" for i in range(num_regions)],
    "医疗设施数": np.random.poisson(lam=10, size=num_regions),
    "养老机构数": np.random.poisson(lam=5, size=num_regions),
    "绿地面积（公顷）": np.random.normal(loc=30, scale=10,
size=num_regions).clip(min=5),
    "文化设施数": np.random.poisson(lam=3, size=num_regions),
    "区域面积（km2）": np.random.uniform(2.0, 20.0,
size=num_regions).round(2),
    "平均寿命（岁）": np.random.normal(loc=76, scale=3,
size=num_regions).round(1),
    "慢性病患率（%）": np.random.normal(loc=15, scale=5,
size=num_regions).clip(min=5, max=40).round(1),
    "健康满意度": np.random.choice(satisfaction_levels, size=num_regions,
p=probabilities)
```

```

}

# 创建 DataFrame
df = pd.DataFrame(data)

# 计算密度类指标
df["医疗设施密度"] = (df["医疗设施数"] / df["区域面积 (km²)"]).round(2)
df["养老机构密度"] = (df["养老机构数"] / df["区域面积 (km²)"]).round(2)
df["绿地密度"] = (df["绿地面积 (公顷)"] / df["区域面积 (km²)"]).round(2)
df["文化设施密度"] = (df["文化设施数"] / df["区域面积 (km²)"]).round(2)

# 输出为 CSV 文件
output_file = "某城市康养的模拟数据 4.csv"
df.to_csv(output_file, index=False, encoding="utf-8-sig")
print(f'模拟数据已成功导出为文件: {output_file}')

```

问题一 资源指数综合得分

```

import pandas as pd
import numpy as np

# 读取 CSV 文件
df = pd.read_csv('1 某城市康养的模拟数据 4.csv')

# 查看数据基本信息和前几行
print('数据基本信息: ')
df.info()

print('\n 数据前几行内容: ')
print(df.head().to_csv(sep='\t', na_rep='nan'))

# 提取最后四列数据
last_four_columns = df.iloc[:, -4:]

# 定义熵值法函数
def entropy_weight(data):

```

```

# 数据标准化（最小-最大归一化）
data_normalized = (data - data.min()) / (data.max() - data.min())

# 计算概率矩阵
p = data_normalized / data_normalized.sum()

# 处理可能的零值（避免 log(0)）
p = p.replace(0, 1e-10)

# 计算熵值
e = -1 / np.log(len(data)) * (p * np.log(p)).sum()

# 计算权重
weights = (1 - e) / (1 - e).sum()

return weights

# 使用熵值法计算权重
weights = entropy_weight(last_four_columns)

# 计算综合得分
df['综合得分_熵值法'] = (last_four_columns * weights).sum(axis=1)

# 查看结果
print('熵值法权重:')
for col, weight in zip(last_four_columns.columns, weights):
    print(f'{col}: {weight:.4f}')

print('\n 综合得分:')
print(df[['区域编号', '综合得分_熵值法']])

# 保存结果为 Excel 文件
df.to_excel('C:/Users/21851/Desktop/某城市康养的模拟数据_资源指数综合得分 1.xlsx', index=False)

```

问题一：健康指数模糊综合评价得分

```
# -*- coding: utf-8 -*-

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

# 设置中文字体
plt.rcParams["font.family"] = ["SimHei", "WenQuanYi Micro Hei", "Heiti TC"]
plt.rcParams["axes.unicode_minus"] = False # 解决负号显示问题

# 读取数据
try:
    file_path = '1 某城市康养的模拟数据 4.xlsx'
    df = pd.read_excel(file_path)
    print(f'数据读取成功，共 {len(df)} 行， {len(df.columns)} 列')
except Exception as e:
    print(f'数据读取失败: {e}')
    exit()

# 明确选中的指标（3 个：2 个定量+1 个定性）
selected_cols = ['平均寿命（岁）', '慢性病患率（%）', '健康满意度']
print(f'选择的列: {selected_cols}')

# 数据准备：处理定性指标“健康满意度”
data = df[selected_cols].copy()

# 1. 定性指标编码：将健康满意度字符串转换为数值
satisfaction_mapping = {'差': 0, '较差': 1, '一般': 2, '较好': 3, '好': 4}
data['健康满意度'] = data['健康满意度'].map(satisfaction_mapping)

# 定义评语集（与梯形区间等级对应）
evaluation_levels = ['差', '较差', '一般', '较好', '好']
level_scores = [0, 1, 2, 3, 4] # 评语量化分数
```

2. 代入优化后的梯形区间参数

每个指标的参数严格匹配其数值优劣方向和实际范围

```
membership_params = {  
    # 平均寿命（岁）：数值越高越优，区间右偏（高值对应高等级）  
    '平均寿命（岁）': {  
        '差': [0, 0, 60, 70],          # 70 岁以下：隶属度逐渐下降  
        '较差': [60, 70, 75, 80],     # 70-75 岁：隶属度先升后降  
        '一般': [70, 75, 80, 85],     # 75-80 岁：核心区间  
        '较好': [75, 80, 85, 90],     # 80-85 岁：较高水平  
        '好': [80, 85, 100, 100]      # 85 岁以上：最优区间  
    },  
    # 慢性病患率（%）：数值越低越优，区间左偏（低值对应高等级）  
    '慢性病患率（%）': {  
        '差': [30, 35, 40, 100],      # 30%以上：最差区间  
        '较差': [20, 25, 30, 35],    # 25-30%：较差水平  
        '一般': [10, 15, 20, 25],    # 15-20%：中等水平  
        '较好': [5, 10, 15, 20],     # 10-15%：较好水平  
        '好': [0, 0, 5, 10]          # 10%以下：最优区间  
    },  
    # 健康满意度（编码 0-4）：数值越高越优，适配编码范围  
    '健康满意度': {  
        '差': [0, 0, 1, 1.5],         # 编码 0：隶属度集中在低区间  
        '较差': [0.5, 1, 1, 2],      # 编码 1：核心区间  
        '一般': [1.5, 2, 2, 3],      # 编码 2：核心区间  
        '较好': [2.5, 3, 3, 4],      # 编码 3：较高水平  
        '好': [3.5, 4, 4, 4]         # 编码 4：最优区间  
    }  
}
```

3. 梯形隶属度函数（计算隶属度的核心逻辑）

```
def trapezoidal_membership(x, a, b, c, d):  
    if x < a or x > d:  
        return 0.0  
    elif a <= x < b:  
        return (x - a) / (b - a)      # 上升沿：隶属度随 x 增大而增大
```

```

elif b <= x <= c:
    return 1.0                # 平顶：完全隶属该等级
elif c < x <= d:
    return (d - x) / (d - c)  # 下降沿：隶属度随 x 增大而减小
else:
    return 0.0

# 4. 计算隶属度矩阵（适配所有指标类型）
membership_matrices = []
for col in selected_cols:
    matrix = []
    for value in data[col]:
        # 为每个指标值计算对 5 个等级的隶属度
        row = [trapezoidal_membership(value,
*membership_params[col][grade])
                for grade in evaluation_levels]
        matrix.append(row)
    membership_matrices.append(np.array(matrix))

# 5. 熵值法计算指标权重（基于数值化后的指标数据）
def entropy_weight_method(data):
    # 归一化处理：消除量纲影响
    normalized = (data - data.min()) / (data.max() - data.min())
    normalized = normalized.replace(0, 1e-10) # 避免 log(0)错误
    # 计算概率矩阵
    p = normalized / normalized.sum()
    # 计算熵值与权重
    e = -1 / np.log(len(data)) * (p * np.log(p)).sum()
    weights = (1 - e) / (1 - e).sum()
    return weights

weights = entropy_weight_method(data).values
print("\n 指标权重:")
for col, w in zip(selected_cols, weights):
    print(f"{col}: {w:.4f}")

```

```

# 6. 模糊合成（加权平均法）与综合得分
final_evaluation = np.zeros((len(data), len(evaluation_levels)))
for i, matrix in enumerate(membership_matrices):
    final_evaluation += matrix * weights[i] # 按权重合并隶属度

# 计算综合得分（隶属度×等级分数）
comprehensive_score = np.sum(final_evaluation * np.array(level_scores),
axis=1)
df['模糊综合得分'] = comprehensive_score.round(2)

# 输出结果
print("\n=== 模糊综合评价结果 ===")
print(df[['区域编号'] + selected_cols + ['模糊综合得分']])

# 可视化各区域得分
plt.figure(figsize=(10, 6))
plt.bar(df['区域编号'], df['模糊综合得分'], color='lightgreen')
plt.axhline(y=df['模糊综合得分'].mean(), color='r', linestyle='--', label='平均得分')
plt.xlabel('区域编号')
plt.ylabel('模糊综合得分')
plt.title('各区域康养模糊综合评价得分（修正梯形区间后）')
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig('修正后模糊综合评价得分.png', dpi=300)
plt.show()

```

问题一 皮尔逊相关系数

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import pearsonr

# 设置中文字体支持

```



```

plt.rcParams["font.family"] = ["SimHei", "WenQuanYi Micro Hei", "Heiti TC"]
plt.rcParams["axes.unicode_minus"] = False # 解决负号显示问题

# 数据输入
health = np.array([0.67, 2.78, 1.53, 2.73, 3.43, 3.02, 2.97, 2.82, 2.61, 2.54,
                   3.07, 2.5, 4.38, 4.12, 1.35, 1.92, 2.37, 2.32, 1.13, 2.59])
resource = np.array([0.7938, 1.1041, 1.4212, 1.2822, 3.7490, 7.1494,
                    0.6517, 1.4605, 3.8042, 0.7202, 0.9896,
                    2.9043, 0.6610, 1.0530, 0.9574,
                    1.2785, 0.8075, 0.8020, 0.9943, 0.6284])

# 计算 Pearson 相关系数及 p 值
r, p_value = pearsonr(resource, health) # 将 p 值变量名改为 p_value
print(f'Pearson 相关系数: {r:.4f}, p 值: {p_value:.4f}')

# 绘制散点图展示相关性
plt.figure(figsize=(10, 6))
plt.scatter(resource, health, alpha=0.7, color='skyblue')
plt.title('资源指数与健康指数的相关性')
plt.xlabel('资源指数')
plt.ylabel('健康指数')
plt.grid(True, linestyle='--', alpha=0.7)

# 添加回归线
z = np.polyfit(resource, health, 1)
p_line = np.poly1d(z) # 将回归线变量名改为 p_line, 避免与 p 值冲突
plt.plot(resource, p_line(resource), "r--")

# 在图表上显示统计信息（使用 p_value 而不是 p）
plt.text(5, 1.5, f'相关系数 r = {r:.4f}\np 值 = {p_value:.4f}', fontsize=12)

plt.tight_layout()
plt.show()

```

问题一 模型优化

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize
import warnings

# 忽略 matplotlib 字体警告
warnings.filterwarnings("ignore", category=UserWarning, module="matplotlib")

# 设置中文字体支持
try:
    plt.rcParams["font.family"] = "SimHei"
    plt.rcParams["axes.unicode_minus"] = False
except:
    print("警告： 无法设置中文字体，图表中的中文可能显示异常")

# 数据输入
health = np.array([0.67, 2.78, 1.53, 2.73, 3.43, 3.02, 2.97, 2.82, 2.61, 2.54,
                  3.07, 2.5, 4.38, 4.12, 1.35, 1.92, 2.37, 2.32, 1.13, 2.59])
resource = np.array([0.7938, 1.1041, 1.4212, 1.2822, 3.7490, 7.1494,
                   0.6517, 1.4605, 3.8042, 0.7202, 0.9896,
                   2.9043, 0.6610, 1.0530, 0.9574,
                   1.2785, 0.8075, 0.8020, 0.9943, 0.6284])

# 数据预处理
def detect_outliers(data, threshold=3):
    z_scores = np.abs((data - np.mean(data)) / np.std(data))
    return z_scores > threshold

outlier_mask = detect_outliers(resource)
print(f'检测到 {np.sum(outlier_mask)} 个异常值: {np.where(outlier_mask)[0] + 1}')

filtered_resource = resource[~outlier_mask]
filtered_health = health[~outlier_mask]
```

```

# 拟合多项式模型
def fit_polynomial(x, y, degree):
    coeffs = np.polyfit(x, y, degree)
    p = np.poly1d(coeffs)
    y_pred = p(x)
    r_squared = 1 - np.sum((y - y_pred) ** 2) / np.sum((y - np.mean(y)) ** 2)
    return coeffs, r_squared

best_degree = 5 # 假设最佳阶数为 5
coeffs, r2 = fit_polynomial(filtered_resource, filtered_health, best_degree)
print(f"最佳拟合模型: {best_degree}阶多项式")
print(f"最佳 R²值: {r2:.6f}")
print(f"最佳系数: {coeffs}")

# 健康指数模型
def health_model(resource, coeffs):
    return np.polyval(coeffs, resource)

# 基尼系数计算
def gini_coefficient(x):
    x = np.array(x, dtype=np.float64)
    x = x[x > 0]
    n = len(x)
    if n == 0:
        return 0

    x_sorted = np.sort(x)
    index = np.arange(1, n + 1)
    return np.sum((2 * index - n - 1) * x_sorted) / (n * np.sum(x_sorted))

# 改进的目标函数
def constrained_objective(resources, coeffs, weights, fairness_weight=0.3,
min_gini=0.2):
    health_values = np.array([health_model(r, coeffs) for r in resources])

```

```

total_health = np.sum(weights * health_values)
gini = gini_coefficient(health_values)

if gini <= min_gini:
    return -total_health
else:
    return -total_health + fairness_weight * (gini - min_gini)

# 多起点优化
def multi_start_optimization(coeffs, weights, total_resources, n_regions,
                             fairness_weight=0.3, min_gini=0.2,
n_starts=10):
    best_result = None
    best_objective = -np.inf

    # 设置差异化的最小资源约束，将每个区域最小资源改为 0.5
    min_resources = [0.5] * n_regions

    bounds = [(min_res, None) for min_res in min_resources]

    for _ in range(n_starts):
        initial_guess = np.random.rand(n_regions)
        initial_guess = initial_guess * total_resources / np.sum(initial_guess)

        result = minimize(
            constrained_objective,
            initial_guess,
            args=(coeffs, weights, fairness_weight, min_gini),
            method='SLSQP',
            bounds=bounds,
            constraints=[{'type': 'eq', 'fun': lambda x: np.sum(x) -
total_resources}]
        )

        if -result.fun > best_objective:

```

```

        best_objective = -result.fun
        best_result = result.x

    return best_result, best_objective

# 执行优化
total_resources = 20
n_regions = len(health)
region_weights = np.ones(n_regions) # 等权重

# 调整公平性权重和最小基尼系数
optimal_resources, max_objective = multi_start_optimization(
    coeffs, region_weights, total_resources, n_regions,
    fairness_weight=0.3, # 降低公平性权重
    min_gini=0.2        # 设置最小基尼系数阈值
)

# 计算优化后的健康指数
optimal_health = np.array([health_model(r, coeffs) for r in optimal_resources])

# 可视化结果
plt.figure(figsize=(15, 10))

# 资源分配对比
plt.subplot(2,2,1)
plt.bar(np.arange(n_regions) - 0.2, resource, width=0.4, label='优化前')
plt.bar(np.arange(n_regions) + 0.2, optimal_resources, width=0.4, label='优化后')

plt.title('资源分配对比')
plt.xlabel('区域编号')
plt.ylabel('资源量')
plt.legend()
plt.grid(axis='y', alpha=0.5)

# 健康指数对比

```

```

plt.subplot(2,2,2)
plt.bar(np.arange(n_regions) - 0.2, health, width=0.4, label='优化前')
plt.bar(np.arange(n_regions) + 0.2, optimal_health, width=0.4, label='优化后')
plt.title('健康指数对比')
plt.xlabel('区域编号')
plt.ylabel('健康指数')
plt.legend()
plt.grid(axis='y', alpha=0.5)

# 洛伦兹曲线
plt.subplot(2,2,3)
def plot_lorenz(ax, values, label, color):
    sorted_values = np.sort(values)
    cumulative = np.cumsum(sorted_values) / np.sum(sorted_values)
    ax.plot(np.linspace(0, 1, len(cumulative)), cumulative, color,
            label=f'{label} (Gini={gini_coefficient(values):.4f})')

plot_lorenz(plt.gca(), health, '优化前', 'r-')
plot_lorenz(plt.gca(), optimal_health, '优化后', 'b-')
plt.plot([0, 1], [0, 1], 'k--', label='绝对公平线')
plt.title('健康指数公平性分析')
plt.xlabel('累积区域比例')
plt.ylabel('累积健康指数比例')
plt.legend()
plt.grid(True)

# 健康指数提升分布
plt.subplot(2,2,4)
improvement = [(oh - h) / h * 100 for oh, h in zip(optimal_health, health)]
plt.bar(range(n_regions), improvement)
plt.title('各区域健康指数提升百分比')
plt.xlabel('区域编号')
plt.ylabel('提升百分比 (%)')
plt.grid(axis='y', alpha=0.5)

```

```

plt.tight_layout()
plt.show()

# 输出结果
print("\n 优化结果:")
print(f'总资源量: {total_resources}')
print(f'优化前总健康指数: {np.sum(health):.4f}')
print(f'优化后总健康指数: {np.sum(optimal_health):.4f}')
print(f'健康指数提升: {((np.sum(optimal_health) - np.sum(health)) /
np.sum(health)) * 100:.2f}%')
print(f'优化前基尼系数: {gini_coefficient(health):.4f}')
print(f'优化后基尼系数: {gini_coefficient(optimal_health):.4f}')

# 输出各区域分配情况
print("\n 各区域资源分配详情:")
for i in range(n_regions):
    print(f'区域 {i+1}: 资源从 {resource[i]:.4f} →
{optimal_resources[i]:.4f} | 健康指数从 {health[i]:.4f} → {optimal_health[i]:.4f}')

```

问题二 数据处理

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. 读取数据并重命名
df = pd.read_excel('/mnt/data/2.xlsx')
df.columns = ['Rank', 'City', 'ResourceIndex', 'LifeExpectancy', 'PM25', 'GDP']

# 2. 归一化
df['R_norm'] = (df['ResourceIndex'] - df['ResourceIndex'].min()) /
(df['ResourceIndex'].max() - df['ResourceIndex'].min())
df['H_norm'] = (df['LifeExpectancy'] - df['LifeExpectancy'].min()) /
(df['LifeExpectancy'].max() - df['LifeExpectancy'].min())
df['G_norm'] = (df['GDP'] - df['GDP'].min()) / (df['GDP'].max() -
df['GDP'].min())

```

```
df['E_norm'] = (df['PM25'].max() - df['PM25']) / (df['PM25'].max() - df['PM25'].min())
```

问题二 权重计算

```
import pandas as pd
import numpy as np

# 加载数据
file_path = 'C:/Users/86133/Desktop/新建文件夹/2.xlsx'
df = pd.read_excel(file_path, sheet_name='_____')

# 选择需要的指标列
data = df[['康养指数', '居民健康状况', '环境质量', '经济发展水平（gdp）']]

# Step 1: 标准化数据，使用 Min-Max 标准化
normalized_data = (data - data.min()) / (data.max() - data.min())

# 输出标准化后的数据
print("标准化后的数据：")
print(normalized_data)

# Step 2: 计算每个值的比例
p = normalized_data / normalized_data.sum(axis=0)

# 输出比例
print("每个值的比例：")
print(p)

# Step 3: 计算熵值
epsilon = 1e-10 # 防止 log(0)错误
entropy = -np.sum(p * np.log(p + epsilon), axis=0) / np.log(len(normalized_data))

# 输出熵值
print("熵值（Entropy）：")
print(entropy)

# Step 4: 计算多样性度量
diversity = 1 - entropy
```



```

# 输出多样性度量
print("多样性度量 (Diversity) : ")
print(diversity)
# Step 5: 计算权重
weights = diversity / diversity.sum()

# 输出权重
print("指标权重: ")
print(weights)
# 假设这是根据 Saaty 标度填充的判断矩阵
A = np.array([
    [1, 3, 5, 7], # 康养指数相对于其他三个指标的比较
    [1/3, 1, 3, 5], # 居民健康状况相对于其他三个指标的比较
    [1/5, 1/3, 1, 3], # 环境质量相对于其他三个指标的比较
    [1/7, 1/5, 1/3, 1], # 经济发展水平相对于其他三个指标的比较
])

# Step 1: 计算每列的和
column_sum = np.sum(A, axis=0)

# Step 2: 归一化矩阵 (每列除以该列的和)
normalized_matrix = A / column_sum

# Step 3: 计算每行的平均值, 这些即为各个指标的权重
weights = np.mean(normalized_matrix, axis=1)

# 输出权重
print("各个指标的权重: ")
print(weights)

# 一致性检验
# Step 4: 计算一致性指标 CI
lambda_max = np.max(np.linalg.eigvals(A)) # 特征值最大值
n = A.shape[0]

```

```
CI = (lambda_max - n) / (n - 1)
```

```
# Step 5: 计算一致性比率 CR
```

```
RI = [0, 0, 0.58, 0.90, 1.12, 1.24, 1.32, 1.41, 1.45] # 不同阶矩阵的随机一致性指标
```

```
CR = CI / RI[n-1]
```

```
# 输出一致性比率
```

```
print("一致性指标 CI:", CI)
```

```
print("一致性比率 CR:", CR)
```

```
# 熵权法权重
```

```
entropy_weights = np.array([0.1523, 0.1710, 0.0516, 0.6251])
```

```
# 层次分析法权重
```

```
ahp_weights = np.array([0.5579, 0.2633, 0.1219, 0.0569])
```

```
# 设置加权平均系数（50%）
```

```
alpha = 0.5
```

```
# 计算综合权重
```

```
combined_weights = alpha * entropy_weights + (1 - alpha) * ahp_weights
```

```
combined_weights
```

问题二 TOPSIS

```
import pandas as pd
```

```
import numpy as np
```

```
# 读取数据
```

```
data_final = pd.read_excel('C:/Users/86133/Desktop/新建文件夹/2.xlsx',  
sheet_name='_____')
```

```
# 提取需要的列：康养指数、居民健康状况、环境质量、经济发展水平
```

```
data_final_filtered = data_final[['康养指数', '居民健康状况', '环境质量', '经济发展水平 (gdp) ']]
```

```

# 给定的权重
weights = np.array([0.3551, 0.2172, 0.0868, 0.3410])

# Step 1: 标准化决策矩阵（使用欧几里得标准化方法）
norm_data = np.sqrt(np.sum(data_final_filtered**2, axis=0))
normalized_matrix = data_final_filtered / norm_data

# Step 2: 加权标准化决策矩阵
weighted_matrix = normalized_matrix * weights

# Step 3: 计算理想解和负理想解
ideal_solution = np.max(weighted_matrix, axis=0)
negative_ideal_solution = np.min(weighted_matrix, axis=0)

# Step 4: 计算每个选项与理想解和负理想解的距离
distance_positive = np.sqrt(np.sum((weighted_matrix - ideal_solution)**2,
axis=1))
distance_negative = np.sqrt(np.sum((weighted_matrix -
negative_ideal_solution)**2, axis=1))

# Step 5: 计算综合得分
C_i = distance_negative / (distance_positive + distance_negative)

# Step 6: 排序综合得分
ranking = np.argsort(C_i)[::-1] # 按得分降序排序

# 输出前 10 个数据的综合得分和排名
print("前 10 个数据的排名和综合得分:")
print("排名:", ranking[:10])
print("综合得分:", C_i[ranking[:10]])

```

问题二 灰色关联分析

```

import pandas as pd
import numpy as np

```

```

# —— 1. 读取数据 ——
# 假设 Excel 中对应列名为 “城市”、“康养指数”、“居民健康状况”、
“环境质量”、“经济发展水平（gdp）”
df = pd.read_excel('C:/Users/86133/Desktop/A2025 材料/2.xlsx',
                  usecols=['城市', '康养指数', '居民健康状况', '环境质量',
'经济发展水平（gdp）'])
df.columns = ['city', 'resource', 'health', 'environment', 'economy']

# —— 2. 数据归一化（极大值 - 极小值规范化，受益型指标） ——
norm_df = df[['resource', 'health', 'environment', 'economy']].apply(
    lambda x: (x - x.min()) / (x.max() - x.min())
)

# —— 3. 构造参考序列（全 1 向量） ——
ref = np.ones(norm_df.shape[1])

# —— 4. 计算各指标与参考序列的绝对差矩阵 ——
diff = np.abs(norm_df.values - ref)

# —— 5. 计算灰色关联系数 ——
min_diff = diff.min()
max_diff = diff.max()
zeta = 0.5 # 分辨系数，一般取 0.5
coeff = (min_diff + zeta * max_diff) / (diff + zeta * max_diff)

# —— 6. 加权计算灰色关联度（关联度等级） ——
weights = np.array([0.3551, 0.2172, 0.0868, 0.3410]) # 各维度权重
grades = coeff.dot(weights)

# —— 7. 将结果合并回原表 ——
df['grade'] = grades
df['rank'] = df['grade'].rank(ascending=False, method='min')

# —— 8. 排序并输出 ——
df_sorted = df.sort_values('grade', ascending=False)

```

```
# 在控制台打印前十
print(df_sorted[['city', 'grade', 'rank']].head(10))

# —— 9. 导出完整结果到 CSV ——
df_sorted.to_csv('grey_relational_grades.csv', index=False, encoding='utf-8-sig')
print("已导出: grey_relational_grades.csv")
```

问题三 粒子群算法

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import random
from typing import Dict, List, Tuple

# 配置中文字体
plt.rcParams["font.family"] = ["SimHei", "WenQuanYi Micro Hei", "Heiti TC"]
plt.rcParams["axes.unicode_minus"] = False

class FacilityLocationOptimizer:
    def __init__(self, df: pd.DataFrame, pop_size: int = 40, max_iter: int = 100):
        """康养设施选址优化器"""
        self.df = df
        self.num_regions = len(df)
        self.distance_matrix = self._compute_distance_matrix()

        # PSO 参数
        self.pop_size = pop_size
        self.max_iter = max_iter
        self.w = 0.7 # 惯性权重
        self.c1 = 1.5 # 个体学习因子
        self.c2 = 1.5 # 社会学习因子

        # 结果存储
```

```

self.best_solution = None
self.best_fitness = -float('inf')
self.fitness_history = []

def _compute_distance_matrix(self) -> np.ndarray:
    """计算区域间距离矩阵"""
    matrix = np.zeros((self.num_regions, self.num_regions))
    for i in range(self.num_regions):
        for j in range(i + 1, self.num_regions):
            lat1, lon1 = self.df.iloc[i][['Latitude', 'Longitude']]
            lat2, lon2 = self.df.iloc[j][['Latitude', 'Longitude']]
            matrix[i, j] = np.sqrt((lat1 - lat2) ** 2 + (lon1 - lon2) ** 2) *
111
            matrix[j, i] = matrix[i, j]
    return matrix

def initialize_particles(self) -> Tuple[List[Dict], List[Dict]]:
    """初始化粒子群及其速度"""
    particles = []
    velocities = []

    for _ in range(self.pop_size):
        # 随机生成固定设施位置（3-7 个）
        fixed_count = random.randint(3, 7)
        fixed_facilities = np.zeros(self.num_regions, dtype=int)
        fixed_indices = random.sample(range(self.num_regions),
fixed_count)
        fixed_facilities[fixed_indices] = 1

        # 随机生成流动车位置（2-5 个，与固定设施不重叠）
        mobile_count = random.randint(2, 5)
        mobile_units = np.zeros(self.num_regions, dtype=int)
        available_indices = [i for i in range(self.num_regions) if not
fixed_facilities[i]]
        mobile_indices = random.sample(available_indices,

```

```

mobile_count)

        mobile_units[mobile_indices] = 1

        # 初始化服务分配矩阵
        service_allocation =

self._initialize_service_matrix(fixed_facilities, mobile_units)

        # 初始化设施规模
        facility_scale = self._initialize_facility_scale(fixed_facilities)

        # 粒子位置
        position = {
            'fixed_facilities': fixed_facilities,
            'mobile_units': mobile_units,
            'service_allocation': service_allocation,
            'facility_scale': facility_scale,
        }

        # 粒子速度
        velocity = {
            'fixed_facilities': np.random.randn(self.num_regions) * 0.1,
            'mobile_units': np.random.randn(self.num_regions) * 0.1,
            'service_allocation': np.random.randn(self.num_regions,
self.num_regions) * 0.05,
            'facility_scale': np.random.randn(self.num_regions) * 0.2,
        }

        particles.append(position)
        velocities.append(velocity)

    return particles, velocities

    def _initialize_service_matrix(self, fixed: np.ndarray, mobile: np.ndarray)
-> np.ndarray:
        """初始化服务分配矩阵"""

```

```

        service = np.zeros((self.num_regions, self.num_regions))
        for i in range(self.num_regions):
            facilities = np.logical_or(fixed, mobile)
            if np.sum(facilities) > 0:
                distances = self.distance_matrix[i, :]
                distances[~facilities] = np.inf
                weights = 1 / (distances + 1e-8) # 距离倒数作为权重
                weights[~facilities] = 0
                if np.sum(weights) > 0:
                    service[i, :] = weights / np.sum(weights)
        return service

    def _initialize_facility_scale(self, fixed_facilities: np.ndarray) ->
np.ndarray:
        """初始化设施规模变量"""
        scale = np.zeros(self.num_regions)
        for i in range(self.num_regions):
            if fixed_facilities[i]:
                # 根据区域人口和慢性病率确定初始规模
                scale[i] = max(30, int(self.df.iloc[i]['Population'] *
self.df.iloc[i]['Chronic_Rate'] * 0.0015))
        return scale

    def evaluate_fitness(self, solution: Dict) -> float:
        """评估解的适应度值（综合考虑覆盖率、成本和利用率）"""
        coverage = self._calculate_coverage(solution)
        cost = self._calculate_cost(solution)
        utilization = self._calculate_utilization(solution)

        # 加权求和
        fitness = 0.5 * coverage + 0.3 * (1 - cost / 2500) + 0.2 * utilization
        return fitness

    def _calculate_coverage(self, solution: Dict) -> float:
        """计算服务覆盖率"""

```



```

        fixed = solution['fixed_facilities']
        mobile = solution['mobile_units']
        service = solution['service_allocation']

        region_coverage = np.zeros(self.num_regions)
        for i in range(self.num_regions):
            # 固定设施覆盖（距离 15 公里内）
            fixed_coverage = sum(service[i, j] for j in
range(self.num_regions)
                                if fixed[j] and self.distance_matrix[i, j]
<= 15)

            # 流动车覆盖（距离 10 公里内）
            mobile_coverage = sum(service[i, k] for k in
range(self.num_regions)
                                if mobile[k] and
self.distance_matrix[i, k] <= 10)

            # 综合覆盖
            region_coverage[i] = min(fixed_coverage + mobile_coverage,
1.0)

        # 加权平均覆盖率（人口和慢性病率作为权重）
        demand_weights = self.df['Population'] * self.df['Chronic_Rate'] / 100
        return np.sum(region_coverage * demand_weights) /
np.sum(demand_weights)

def _calculate_cost(self, solution: Dict) -> float:
    """计算总成本"""
    fixed = solution['fixed_facilities']
    mobile = solution['mobile_units']
    scale = solution['facility_scale']

    # 建设成本（百万元）
    construction_cost = sum(fixed[i] * (150 + scale[i] * 4) for i in

```

```

range(self.num_regions)) / 100

        # 运营成本（百万元/年）
        operation_cost = sum(fixed[i] * (25 + scale[i] * 0.8) for i in
range(self.num_regions)) / 100
        operation_cost += np.sum(mobile) * 18 # 每辆流动车年运营成本
18 万元

        return construction_cost + operation_cost

def _calculate_utilization(self, solution: Dict) -> float:
    """计算资源利用率"""
    fixed = solution['fixed_facilities']
    service = solution['service_allocation']
    scale = solution['facility_scale']

    # 计算总需求
    total_demand = self.df['Population'] * self.df['Chronic_Rate'] / 100

    # 计算每个设施的利用率
    facility_utilization = np.zeros(self.num_regions)
    for j in range(self.num_regions):
        if fixed[j]:
            served = sum(service[i, j] * total_demand[i] for i in
range(self.num_regions))
            facility_utilization[j] = min(served / (scale[j] * 365 * 0.7),
1.0) # 设施容量利用率

    # 计算平均利用率
    return np.mean(facility_utilization[fixed == 1]) if np.sum(fixed) > 0
else 0

def update_particles(self, particles: List[Dict], velocities: List[Dict],
                    personal_best: List[Dict], personal_best_fitness:
List[float]) -> None:

```

```

        """更新粒子位置和速度"""
        for i in range(self.pop_size):
            new_position = {}

            # 更新每个维度的位置和速度
            for key in ['fixed_facilities', 'mobile_units', 'service_allocation',
'facility_scale']:

                r1, r2 = random.random(), random.random()

                # 个体学习部分
                cognitive = self.c1 * r1 * (personal_best[i][key] -
particles[i][key])

                # 社会学习部分
                social = self.c2 * r2 * (self.best_solution[key] -
particles[i][key])

                # 更新速度
                velocities[i][key] = self.w * velocities[i][key] + cognitive +
social

                # 速度限制
                if key in ['fixed_facilities', 'mobile_units']:
                    velocities[i][key] = np.clip(velocities[i][key], -0.12,
0.12)

                elif key == 'facility_scale':
                    velocities[i][key] = np.clip(velocities[i][key], -4, 4)
                else:
                    velocities[i][key] = np.clip(velocities[i][key], -0.04,
0.04)

                # 更新位置
                new_position[key] = particles[i][key] + velocities[i][key]

            # 边界处理

```

```

        if key in ['fixed_facilities', 'mobile_units']:
            new_position[key] = np.clip(new_position[key], 0,
1).astype(int)

        elif key == 'facility_scale':
            new_position[key] = np.clip(new_position[key], 20,
250).astype(int)

        # 处理固定设施和流动车的重叠问题
        overlap = np.logical_and(new_position['fixed_facilities'],
new_position['mobile_units'])
        if np.any(overlap):
            overlap_indices = np.where(overlap)[0]
            for idx in overlap_indices:
                # 随机选择保留固定设施或流动车
                choice = random.choice(['fixed_facilities',
'mobile_units'])

                new_position[choice][idx] = 1
                new_position['fixed_facilities' if choice ==
'mobile_units' else 'mobile_units'][idx] = 0

        # 重新计算服务分配矩阵
        new_position['service_allocation'] =
self._initialize_service_matrix(
            new_position['fixed_facilities'], new_position['mobile_units']
        )

        # 评估新位置的适应度
        fitness = self.evaluate_fitness(new_position)

        # 更新个体最优
        if fitness > personal_best_fitness[i]:
            personal_best_fitness[i] = fitness
            personal_best[i] = new_position.copy()

        # 更新全局最优

```

```

        if fitness > self.best_fitness:
            self.best_fitness = fitness
            self.best_solution = new_position.copy()

        # 更新粒子位置
        particles[i] = new_position

    # 记录当前迭代的最佳适应度
    self.fitness_history.append(self.best_fitness)

def optimize(self) -> None:
    """执行粒子群优化算法"""
    print("开始康养设施选址优化...")

    # 初始化粒子群和速度
    particles, velocities = self.initialize_particles()

    # 初始化个体最优
    personal_best = [p.copy() for p in particles]
    personal_best_fitness = [self.evaluate_fitness(p) for p in particles]

    # 初始化全局最优
    best_idx = np.argmax(personal_best_fitness)
    self.best_fitness = personal_best_fitness[best_idx]
    self.best_solution = personal_best[best_idx].copy()

    # 迭代优化
    for iteration in range(self.max_iter):
        self.update_particles(particles, velocities, personal_best,
personal_best_fitness)

        # 每 20 次迭代打印进度
        if (iteration + 1) % 20 == 0:
            print(f'迭代 {iteration + 1}/{self.max_iter}, 最佳适应度:
{self.best_fitness:.4f}')

```

```

print("优化完成!")

def visualize_solution(self) -> None:
    """可视化优化结果"""
    if self.best_solution is None:
        print("请先运行优化算法!")
        return

    plt.figure(figsize=(12, 8))

    # 绘制区域分布
    scatter = plt.scatter(
        self.df['Longitude'], self.df['Latitude'],
        s=self.df['Population'] / 300, # 点大小表示人口
        c=self.df['Chronic_Rate'], # 颜色表示慢性病率
        cmap='Reds', alpha=0.7, edgecolors='k'
    )

    # 标记固定设施（大小表示规模）
    fixed_indices = np.where(self.best_solution['fixed_facilities'] == 1)[0]
    plt.scatter(
        self.df.iloc[fixed_indices]['Longitude'],
        self.df.iloc[fixed_indices]['Latitude'],
        s=self.best_solution['facility_scale'][fixed_indices] * 3, # 设施
        c='blue', marker='^', label='固定设施'
    )

    # 标记流动车
    mobile_indices = np.where(self.best_solution['mobile_units'] == 1)[0]
    plt.scatter(
        self.df.iloc[mobile_indices]['Longitude'],
        self.df.iloc[mobile_indices]['Latitude'],
        s=180, c='green', marker='s', label='流动车'
    )

```

```

)

# 添加标题和图例
plt.title('康养设施优化配置方案', fontsize=16)
plt.xlabel('经度', fontsize=12)
plt.ylabel('纬度', fontsize=12)
plt.colorbar(scatter, label='慢性病率 (%)')
plt.legend(loc='upper right')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

def print_decision_variables(self) -> None:
    """打印所有决策变量的具体数值"""
    if self.best_solution is None:
        print("请先运行优化算法!")
        return

    print("\n=== 康养设施优化配置决策变量 ===")

    # 1. 设施选址变量
    print("\n1. 设施选址变量:")
    fixed_indices = np.where(self.best_solution['fixed_facilities'] == 1)[0]
    for i, idx in enumerate(fixed_indices, 1):
        region = self.df.iloc[idx]
        scale = self.best_solution['facility_scale'][idx]
        print(f"    设施 {i}:")
        print(f"        - 区域: {region['Region']}")
        print(f"        - 坐标 : ({region['Longitude']:.4f}, {region['Latitude']:.4f})")
        print(f"        - 人口: {region['Population']}")
        print(f"        - 慢性病率: {region['Chronic_Rate']:.1f}%")
        print(f"        - 设施规模: {scale} 床位")

    # 2. 流动车部署变量

```

```

print("\n2. 流动车部署变量:")
mobile_indices = np.where(self.best_solution['mobile_units'] == 1)[0]
for i, idx in enumerate(mobile_indices, 1):
    region = self.df.iloc[idx]
    print(f"    流动车 {i}:")
    print(f"        - 部署区域: {region['Region']}")
    print(f"        - 坐 标 : ({region['Longitude']:.4f},
{region['Latitude']:.4f})")
    print(f"        - 服务人口: {region['Population']}")
    print(f"        - 区域慢性病率: {region['Chronic_Rate']:.1f}%")

# 3. 服务量分配变量
print("\n3. 服务量分配变量 (前 10 个区域):")
service = self.best_solution['service_allocation']
facilities = np.logical_or(self.best_solution['fixed_facilities'],
                           self.best_solution['mobile_units'])
facility_indices = np.where(facilities)[0]

# 打印表头
header = "需求区域".ljust(8)
for idx in facility_indices:
    facility_type = "固定" if self.best_solution['fixed_facilities'][idx]
else "流动"
    header +=
f"{self.df.iloc[idx]['Region']}({facility_type}).ljust(15)
print(header)

# 打印服务分配矩阵
for i in range(min(10, self.num_regions)):
    row = f"{self.df.iloc[i]['Region']}.ljust(8)
    for idx in facility_indices:
        row += f"{service[i, idx]:.4f}.ljust(15)
    print(row)

if self.num_regions > 10:

```



```

        print(f"... 显示前 10 个区域，共 {self.num_regions} 个区域")

# 4. 设施规模变量
print("\n4. 设施规模变量:")
for i, idx in enumerate(fixed_indices, 1):
    region = self.df.iloc[idx]
    scale = self.best_solution['facility_scale'][idx]
    print(f"    设施 {i} ({region['Region']}): {scale} 床位")

# 5. 关键指标
coverage = self._calculate_coverage(self.best_solution)
cost = self._calculate_cost(self.best_solution)
utilization = self._calculate_utilization(self.best_solution)

print("\n5. 关键指标:")
print(f"    - 服务覆盖率: {coverage:.2%}")
print(f"    - 总成本: {cost:.2f} 百万元")print(f"    - 资源利用率: {utilization:.2%}")

# 生成模拟数据（保持 20 个区域，但调整分布）
np.random.seed(42) # 设置随机种子，确保结果可复现
data = {
    'Region': [f'R{i}' for i in range(1, 21)],
    'Latitude': np.random.uniform(30.5, 31.5, 20).round(4),
    'Longitude': np.random.uniform(120.5, 121.5, 20).round(4),
    'Population': np.random.randint(8000, 35000, 20), # 调整人口范围
    'Chronic_Rate': np.random.uniform(12.0, 28.0, 20).round(1) # 调整慢性病率范围
}
df = pd.DataFrame(data)

# 运行优化
optimizer = FacilityLocationOptimizer(df, pop_size=40, max_iter=200)
optimizer.optimize()

```

```
optimizer.visualize_solution()  
optimizer.print_decision_variables()
```